

Human review packet: MendelsonWhang1990PriorityPricing

Generated from byte-pinned source excerpts, the expanded PaperInterface specifications, and hash-bound raw-source-to-Spec screening records.

Paper: MendelsonWhang1990PriorityPricing

Paper id: MendelsonWhang1990PriorityPricing

Source version: Operations Research 38(5), September–October 1990, pp. 870–883

Source record: audit/paper_statement_map.json

Rows: 5

Generated: 2026-09-07

Source URL: [official source](#)

How to use this packet

For each source claim, compare the exact source excerpts with the expanded PaperInterface specification. Mark up this PDF or fill the blank reviewer box in the TeX source; return the annotated file for reconciliation into the paper-local human-review log.

Open the interactive dashboard

The interactive dashboard is an optional alternative to using this PDF; it presents the same review surface rather than an additional review requirement. After forking and cloning (or pulling) AppliedModelingLib, run the following from the repository root. The cache command is needed only the first time in a new clone.

```
cd <your-repository-checkout>
lake exe cache get
python3 scripts/review_dashboard.py --paper MendelsonWhang1990PriorityPricing --serve
```

Open <http://127.0.0.1:8765/> in a browser and keep that terminal running while reviewing. The dashboard records saved annotations in this paper's local reviewer-owned review record.

Contents

- [Governing Lean declarations \(86; supporting context\)](#)
- [Semantic library prerequisites \(23; not paper claims\)](#)
 - [AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkMeasure](#)
 - [AppliedModelingLib.Probability.stabilizedFiniteReplayResponse](#)
 - [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
 - [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
 - [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
 - [AppliedModelingLib.Queueing.admitNonpreemptivePriorityJob](#)
 - [AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState](#)
 - [AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs](#)
 - [AppliedModelingLib.Queueing.runNonpreemptivePriorityArrivalTrace](#)
 - [AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFiniteWindowState](#)
 - [AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFutureArrivalJobs](#)
 - [AppliedModelingLib.Queueing.clearNonpreemptivePriorityCompletionLedger](#)
 - [AppliedModelingLib.Queueing.finiteNonpreemptivePriorityQueueWait](#)
 - [AppliedModelingLib.Queueing.nonpreemptivePriorityRecordedCompletionTime](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedJob](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFiniteReplayPostArrivalState](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFiniteReplayPostArrivalResponseTime](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedPalmMeasure](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedResponseTime](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirement](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedQueueWait](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityWorkRequirement](#)
 - [AppliedModelingLib.Queueing.stationaryPriorityWorkRequirement_hasLaw](#)
- [Paper-specific semantic prerequisites \(19; not paper claims\)](#)
 - [MendelsonWhang1990PriorityPricing.priorityTimeDependentPrice](#)

- MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient
- MendelsonWhang1990PriorityPricing.priorityTimeQuadraticCoefficient
- MendelsonWhang1990PriorityPricing.heterogeneousPriorityTimeDependentPrice
- MendelsonWhang1990PriorityPricing.priorityQueueingTime
- MendelsonWhang1990PriorityPricing.homogeneousPriorityQueueingTime
- MendelsonWhang1990PriorityPricing.homogeneousPriorityPrice
- MendelsonWhang1990PriorityPricing.homogeneousPrioritySojournTime
- MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost
- MendelsonWhang1990PriorityPricing.homogeneousPriorityPopulation
- MendelsonWhang1990PriorityPricing.individualDemandCondition
- MendelsonWhang1990PriorityPricing.netValue
- MendelsonWhang1990PriorityPricing.priorityCheatingPenalty
- MendelsonWhang1990PriorityPricing.priorityTimeDependentExpectedCost
- MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain
- MendelsonWhang1990PriorityPricing.priorityTimeDependentPricingOptimalAndIncentiveCompatibleAt
- MendelsonWhang1990PriorityPricing.sourceDelayCostConditions
- MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions
- MendelsonWhang1990PriorityPricing.stationaryClassTaggedExpectedPriorityTimeDependentPrice
- Source claims (5)
 - theoremOneExternalityPricingSpec
 - appendixPriorityQueueingTimeSpec
 - theoremTwoPriorityIncentiveCompatibilitySpec
 - theoremThreePriorityTimeDependentPricingSpec
 - theoremFourCheatingPenaltyMonotonicitySpec

Governing Lean declarations

These exact graph-bound declaration bodies are retained by the displayed specifications or reviewed prerequisites. They are supporting context and do not add source-result rows or semantic judgments.

AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw

Declaration location: reusable library

Lean declaration body

```
field Pbase:
{Ω : Type u_1} →
[inst : MeasurableSpace Ω] →
AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Ω → MeasureTheory.Measure Ω

field isProbability:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (self : AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Ω),
MeasureTheory.IsProbabilityMeasure self.Pbase

field shift:
{Ω : Type u_1} →
[inst : MeasurableSpace Ω] → AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Ω → ℝ → Ω → Ω

field shift_zero:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (self : AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Ω),
self.shift 0 = id

field shift_add:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (self : AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Ω)
(s t : ℝ), self.shift (s + t) = self.shift s ∘ self.shift t

field shift_preserving:
∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (self : AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Ω)
(t : ℝ), MeasureTheory.MeasurePreserving (self.shift t) self.Pbase self.Pbase
```

AppliedModelingLib.Probability.PoissonProcess.interarrival

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → (ℕ → ℝ) → ℝ

value:
fun (n : ℕ) (a : ℕ → ℝ) => a n
```

AppliedModelingLib.Probability.PoissonProcess.arrivalTime

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → (ℕ → ℝ) → ℝ

value:
fun (n : ℕ) (a : ℕ → ℝ) => ∑ i ∈ Finset.range (n + 1), AppliedModelingLib.Probability.PoissonProcess.interarrival i a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.interarrival](#)

AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → (ℕ → ℝ) → ℕ

value:
fun (t : ℝ) (a : ℕ → ℝ) =>
if h : ∃ (n : ℕ), t < AppliedModelingLib.Probability.PoissonProcess.arrivalTime n a then Nat.find h else 0
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.arrivalTime](#)

AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCountLE

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow (\mathbb{N} \rightarrow \mathbb{R}) \rightarrow \mathbb{N}$ 

value:
fun (t :  $\mathbb{R}$ ) (a :  $\mathbb{N} \rightarrow \mathbb{R}$ ) =>
  if h :  $\exists (n : \mathbb{N}), t \leq \text{AppliedModelingLib.Probability.PoissonProcess.arrivalTime } n \text{ a}$  then Nat.find h else 0
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.arrivalTime](#)

AppliedModelingLib.Probability.PoissonProcess.twoSidedGap

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{Z} \rightarrow (\mathbb{Z} \rightarrow \mathbb{R}) \rightarrow \mathbb{R}$ 

value:
fun (i :  $\mathbb{Z}$ ) (a :  $\mathbb{Z} \rightarrow \mathbb{R}$ ) => a i
```

AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival

Declaration location: reusable library

Lean declaration body

```
type:
 $(\mathbb{Z} \rightarrow \mathbb{R}) \rightarrow \mathbb{Z} \rightarrow \mathbb{R}$ 

value:
fun ( $\omega : \mathbb{Z} \rightarrow \mathbb{R}$ ) (a :  $\mathbb{Z}$ ) =>
  match a with
  | Int.ofNat n =>  $\sum i \in \text{Finset.range } n, \text{AppliedModelingLib.Probability.PoissonProcess.twoSidedGap } (\text{Int.ofNat } i) \omega$ 
  | Int.negSucc n =>
     $-\sum i \in \text{Finset.range } (n + 1), \text{AppliedModelingLib.Probability.PoissonProcess.twoSidedGap } (\text{Int.negSucc } i) \omega$ 
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.twoSidedGap](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier

Declaration location: reusable library

Lean declaration body

```
type:
 $(\mathbb{Z} \rightarrow \mathbb{R}) \times \mathbb{R} \rightarrow \text{Prop}$ 

value:
fun (a :  $(\mathbb{Z} \rightarrow \mathbb{R}) \times \mathbb{R}$ ) =>
  {p :  $(\mathbb{Z} \rightarrow \mathbb{R}) \times \mathbb{R} \mid 0 \leq p.2 \wedge p.2 < \text{AppliedModelingLib.Probability.PoissonProcess.twoSidedGap } 0 \text{ p.1} \} a$ 
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.twoSidedGap](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionFuturePath

Declaration location: reusable library

Lean declaration body

```
type:
( $\mathbb{Z} \rightarrow \mathbb{R}$ )  $\rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (a :  $\mathbb{Z} \rightarrow \mathbb{R}$ ) (a_1 :  $\mathbb{N}$ ) => AppliedModelingLib.Probability.PoissonProcess.twoSidedGap (Int.ofNat a_1) a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.twoSidedGap](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionGapShift

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{Z} \rightarrow (\mathbb{Z} \rightarrow \mathbb{R}) \rightarrow \mathbb{Z} \rightarrow \mathbb{R}$ 

value:
fun (k :  $\mathbb{Z}$ ) (a :  $\mathbb{Z} \rightarrow \mathbb{R}$ ) (a_1 :  $\mathbb{Z}$ ) => AppliedModelingLib.Probability.PoissonProcess.twoSidedGap (a_1 + k) a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.twoSidedGap](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionPastPath

Declaration location: reusable library

Lean declaration body

```
type:
( $\mathbb{Z} \rightarrow \mathbb{R}$ )  $\rightarrow \mathbb{N} \rightarrow \mathbb{R}$ 

value:
fun (a :  $\mathbb{Z} \rightarrow \mathbb{R}$ ) (a_1 :  $\mathbb{N}$ ) => AppliedModelingLib.Probability.PoissonProcess.twoSidedGap (Int.negSucc a_1) a
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.twoSidedGap](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow (\mathbb{Z} \rightarrow \mathbb{R}) \times \mathbb{R} \rightarrow \mathbb{Z}$ 

value:
fun (t :  $\mathbb{R}$ ) (a : ( $\mathbb{Z} \rightarrow \mathbb{R}$ )  $\times \mathbb{R}$ ) =>
  if 0 ≤ a.2 + t then
    Int.ofNat
      (AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount (a.2 + t)
        (AppliedModelingLib.Probability.PoissonProcess.suspensionFuturePath a.1))
  else
    Int.negSucc
      (AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCountLE (-(a.2 + t))
        (AppliedModelingLib.Probability.PoissonProcess.suspensionPastPath a.1))
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCount](#)
- [AppliedModelingLib.Probability.PoissonProcess.canonicalRenewalCountLE](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionFuturePath](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionPastPath](#)

AppliedModelingLib.Probability.PoissonProcess.palmTaggedArrivalIndices

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{Z} \rightarrow \mathbb{R}) \rightarrow \text{Finset } \mathbb{Z}$ 

value:
fun (a b :  $\mathbb{R}$ ) (g :  $\mathbb{Z} \rightarrow \mathbb{R}$ ) =>
  {i ∈
    Finset.Icc (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed a (g, 0))
      (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed b (g, 0)) |
    a ≤ AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival g i ∧
    AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival g i < b}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed](#)

AppliedModelingLib.Probability.PoissonProcess.palmTaggedArrivalIndicesRightClosed

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow (\mathbb{Z} \rightarrow \mathbb{R}) \rightarrow \text{Finset } \mathbb{Z}$ 

value:
fun (a b :  $\mathbb{R}$ ) (g :  $\mathbb{Z} \rightarrow \mathbb{R}$ ) =>
  {i ∈
    Finset.Icc (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed a (g, 0))
      (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed b (g, 0)) |
    a < AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival g i ∧
    AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival g i ≤ b}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionFlow

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow (\mathbb{Z} \rightarrow \mathbb{R}) \times \mathbb{R} \rightarrow (\mathbb{Z} \rightarrow \mathbb{R}) \times \mathbb{R}$ 

value:
fun (t :  $\mathbb{R}$ ) (a :  $(\mathbb{Z} \rightarrow \mathbb{R}) \times \mathbb{R}$ ) =>
  have k :  $\mathbb{Z}$  := AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed t a;
  (AppliedModelingLib.Probability.PoissonProcess.suspensionGapShift k a.1,
    a.2 + t - AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival a.1 k)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGapShift](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath

Declaration location: reusable library

Lean declaration body

```
type:
  ( $\mathbb{Z} \rightarrow \mathbb{R}$ )  $\rightarrow$  Prop

value:
  fun (ω :  $\mathbb{Z} \rightarrow \mathbb{R}$ ) =>
    (∀ (i :  $\mathbb{Z}$ ), 0 < AppliedModelingLib.Probability.PoissonProcess.twoSidedGap i ω) ∧
    ∀ (k :  $\mathbb{Z}$ ),
      Filter.Tendsto
        (fun (n :  $\mathbb{N}$ ) =>
          AppliedModelingLib.Probability.PoissonProcess.arrivalTime n
            (AppliedModelingLib.Probability.PoissonProcess.suspensionFuturePath
              (AppliedModelingLib.Probability.PoissonProcess.suspensionGapShift k ω)))
          Filter.atTop Filter.atTop ω
      Filter.Tendsto
        (fun (n :  $\mathbb{N}$ ) =>
          AppliedModelingLib.Probability.PoissonProcess.arrivalTime n
            (AppliedModelingLib.Probability.PoissonProcess.suspensionPastPath
              (AppliedModelingLib.Probability.PoissonProcess.suspensionGapShift k ω)))
          Filter.atTop Filter.atTop ω
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.arrivalTime](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionFuturePath](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGapShift](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionPastPath](#)
- [AppliedModelingLib.Probability.PoissonProcess.twoSidedGap](#)

AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState

Declaration location: reusable library

Lean declaration body

```
type:
  type

value:
  { p : ( $\mathbb{Z} \rightarrow \mathbb{R}$ )  $\times$   $\mathbb{R}$  //
    p ∈ AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier ∧
    AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath p.1 }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)

AppliedModelingLib.Probability.PoissonProcess.goodSuspensionFlow

Declaration location: reusable library

Lean declaration body

```
type:
   $\mathbb{R} \rightarrow$ 
    AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState  $\rightarrow$ 
    AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState

value:
  fun (t :  $\mathbb{R}$ ) (a : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState) =>
    (AppliedModelingLib.Probability.PoissonProcess.suspensionFlow t ↑ a,
     AppliedModelingLib.Probability.PoissonProcess.goodSuspensionFlow._proof_1 t a)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionFlow](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)

AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState

Declaration location: reusable library

Lean declaration body

```
type:
MeasurableSpace AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState

value:
{ MeasurableSet' := AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState._aux_1,
  measurableSet_empty := AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState._proof_3,
  measurableSet_compl := AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState._proof_4,
  measurableSet_iUnion :=
    AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState._proof_5 }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival

Declaration location: reusable library

Lean declaration body

```
type:
AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState →  $\mathbb{Z} \rightarrow \mathbb{R}$ 

value:
fun (p : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState) (i :  $\mathbb{Z}$ ) =>
  AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival (↑p).1 i - (↑p).2
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrivalIndices

Declaration location: reusable library

Lean declaration body

```
type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow$  AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState → Finset  $\mathbb{Z}$ 

value:
fun (a b :  $\mathbb{R}$ ) (p : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState) =>
{ i ∈
  Finset.lcc (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed a ↑p)
  (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed b ↑p) |
  a ≤ AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival p i ∧
  AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival p i < b }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)

AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrivalIndicesRightClosed

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → ℝ → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState → Finset ℤ

value:
fun (a b : ℝ) (p : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState) =>
  {i ∈
    Finset.Icc (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed a ↑ p)
      (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed b ↑ p) |
    a < AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival p i ∧
    AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival p i ≤ b}
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)

AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → MeasureTheory.Measure (ℤ → ℝ)

value:
fun (rate : ℝ) => MeasureTheory.Measure.infinitePi fun (x : ℤ) => ProbabilityTheory.expMeasure rate
```

AppliedModelingLib.Probability.PoissonProcess.suspensionMeasure

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → MeasureTheory.Measure ((ℤ → ℝ) × ℝ)

value:
fun (rate : ℝ) =>
  ENNReal.ofReal rate •
  ((AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure rate).prod
    MeasureTheory.volume).restrict
  AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure](#)

AppliedModelingLib.Probability.PoissonProcess.goodSuspensionMeasure

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → MeasureTheory.Measure AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState

value:
fun (rate : ℝ) =>
  MeasureTheory.Measure.comap Subtype.val (AppliedModelingLib.Probability.PoissonProcess.suspensionMeasure rate)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionMeasure](#)

AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero

Declaration location: reusable library

Lean declaration body

```
field Ptag:
  {Ω : Type u_1} →
  [inst : MeasurableSpace Ω] → AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero Ω → MeasureTheory.Measure Ω

field isProbability:
  ∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (self : AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero Ω),
  MeasureTheory.IsProbabilityMeasure self.Ptag

field arrivals:
  {Ω : Type u_1} → [inst : MeasurableSpace Ω] → AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero Ω → Ω → ℤ → ℝ

field tag_at_zero:
  ∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (self : AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero Ω),
  ∀m (ω : Ω) ∂self.Ptag, self.arrivals ω 0 = 0

field arrivals_strict:
  ∀ {Ω : Type u_1} [inst : MeasurableSpace Ω] (self : AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero Ω),
  ∀m (ω : Ω) ∂self.Ptag, StrictMono (self.arrivals ω)
```

AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero

Declaration location: reusable library

Lean declaration body

```
type:
  {Ωtag : Type u_1} →
  {Γ : Type u_2} →
  [inst : MeasurableSpace Ωtag] →
  [inst_1 : MeasurableSpace Γ] →
  AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero Ωtag →
  AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Γ →
  AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero (Ωtag × Γ)

value:
fun {Ωtag : Type u_1} {Γ : Type u_2} [MeasurableSpace Ωtag] [MeasurableSpace Γ]
  (tagged : AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero Ωtag)
  (passive : AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw Γ) =>
  { Ptag := tagged.Ptag.prod passive.Pbase,
    isProbability := AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero._proof_1 tagged passive,
    arrivals := fun (z : Ωtag × Γ) => tagged.arrivals z.1,
    tag_at_zero := AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero._proof_3 tagged passive,
    arrivals_strict := AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero._proof_4 tagged passive }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw](#)
- [AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero](#)

AppliedModelingLib.Probability.PoissonProcess.candidateTaggedArrivalAtZero

Declaration location: reusable library

Lean declaration body

```
type:
  (rate : ℝ) → 0 < rate → AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero (ℤ → ℝ)

value:
fun (rate : ℝ) (hrate : 0 < rate) =>
  { Ptag := AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure rate,
    isProbability :=
      AppliedModelingLib.Probability.PoissonProcess.isProbabilityMeasure_twoSidedInterarrivalMeasure hrate,
    arrivals := AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival,
    tag_at_zero := AppliedModelingLib.Probability.PoissonProcess.ae_candidatePalmArrival_tag_at_zero,
    arrivals_strict := AppliedModelingLib.Probability.PoissonProcess.ae_candidatePalmArrival_strictMono hrate }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure](#)
- [AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero](#)

AppliedModelingLib.Probability.Queueing.intPathShift

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → ℤ → (ℤ → α) → ℤ → α
value:
fun {α : Type u_1} (k : ℤ) (x : ℤ → α) (a : ℤ) => x (k + a)
```

AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkRequirement

Declaration location: reusable library

Lean declaration body

```
type:
AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ) → ℤ → ℝ
value:
fun (z : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) (i : ℤ) => z.2 i
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)

AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkRequirement

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} →
Class → (Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) → ℤ → ℝ
value:
fun {Class : Type u_1} (i : Class)
(a : Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) (a_1 : ℤ) =>
AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkRequirement (a i) a_1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkRequirement](#)

AppliedModelingLib.Probability.Queueing.timedEmbeddedArrival

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → α) → ℤ → ℝ
value:
fun {α : Type u_1} (z : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → α)) (i : ℤ) =>
AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival z.1 i
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrival](#)

AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkArrival

Declaration location: reusable library

Lean declaration body

```
type:
  AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ) → ℤ → ℝ

value:
  fun (z : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) (i : ℤ) =>
    AppliedModelingLib.Probability.Queueing.timedEmbeddedArrival z i
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.Queueing.timedEmbeddedArrival](#)

AppliedModelingLib.Probability.Queueing.timedEmbeddedSuspensionFlow

Declaration location: reusable library

Lean declaration body

```
type:
  {α : Type u_1} →
  ℝ →
  AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → α) →
  AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → α)

value:
  fun {α : Type u_1} (t : ℝ) (a : AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → α)) =>
    (AppliedModelingLib.Probability.PoissonProcess.goodSuspensionFlow t a.1,
     AppliedModelingLib.Probability.Queueing.intPathShift
      (AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed t ↑a.1) a.2)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.goodSuspensionFlow](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCarrier](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionCrossingIndexPastClosed](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionGoodGapPath](#)
- [AppliedModelingLib.Probability.Queueing.intPathShift](#)

AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkFlow

Declaration location: reusable library

Lean declaration body

```
type:
  {Class : Type u_1} →
  ℝ →
  (Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) →
  Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)

value:
  fun {Class : Type u_1} (t : ℝ) (a : Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ))
    (a_1 : Class) =>
    AppliedModelingLib.Probability.Queueing.timedEmbeddedSuspensionFlow t (a a_1)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.Queueing.timedEmbeddedSuspensionFlow](#)

AppliedModelingLib.Probability.Queueing.timedEmbeddedSuspensionProductMeasure

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
[inst : MeasurableSpace α] →
ℝ →
  MeasureTheory.Measure (ℤ → α) →
  MeasureTheory.Measure (AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → α))

value:
fun {α : Type u_1} [MeasurableSpace α] (rate : ℝ) (P : MeasureTheory.Measure (ℤ → α)) =>
  (AppliedModelingLib.Probability.PoissonProcess.goodSuspensionMeasure rate).prod P
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.goodSuspensionMeasure](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)

AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkMeasure

Declaration location: reusable library

Lean declaration body

```
type:
ℝ → MeasureTheory.Measure (AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ))

value:
fun (arrivalRate : ℝ) =>
  AppliedModelingLib.Probability.Queueing.timedEmbeddedSuspensionProductMeasure arrivalRate
  (AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure 1)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure](#)
- [AppliedModelingLib.Probability.Queueing.timedEmbeddedSuspensionProductMeasure](#)

AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkShiftInvariantLaw

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} →
[Fintype Class] →
(arrivalRate : Class → ℝ) →
(∀ (i : Class), 0 < arrivalRate i) →
  AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw
  (Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ))

value:
fun {Class : Type u_1} [Fintype Class] (arrivalRate : Class → ℝ) (harrivalRate : ∀ (i : Class), 0 < arrivalRate i) =>
{ Pbase := AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkMeasure arrivalRate,
  isProbability :=
    AppliedModelingLib.Probability.PoissonProcess.isProbabilityMeasure_multiclassStationaryPoissonWorkMeasure
    arrivalRate harrivalRate,
  shift := AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkFlow,
  shift_zero :=
    AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkShiftInvariantLaw._proof_1,
  shift_add :=
    AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkShiftInvariantLaw._proof_2,
  shift_preserving := fun (t : ℝ) =>
    AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkFlow_measurePreserving arrivalRate
    harrivalRate t }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw](#)
- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkFlow](#)
- [AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkMeasure](#)

AppliedModelingLib.Probability.Queueing.timedEmbeddedTaggedArrivalAtZero

Declaration location: reusable library

Lean declaration body

```
type:
{α : Type u_1} →
[inst : MeasurableSpace α] →
(rate : ℝ) →
0 < rate →
(PtagPath : MeasureTheory.Measure (ℤ → α)) →
[MeasureTheory.IsProbabilityMeasure PtagPath] →
AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero ((ℤ → ℝ) × (ℤ → α))

value:
fun {α : Type u_1} [MeasurableSpace α] (rate : ℝ) (hrate : 0 < rate) (PtagPath : MeasureTheory.Measure (ℤ → α))
[MeasureTheory.IsProbabilityMeasure PtagPath] =>
{ Ptag := (AppliedModelingLib.Probability.PoissonProcess.candidateTaggedArrivalAtZero rate hrate).Ptag.prod PtagPath,
isProbability :=
AppliedModelingLib.Probability.Queueing.timedEmbeddedTaggedArrivalAtZero._proof_1 rate hrate PtagPath,
arrivals := fun (z : (ℤ → ℝ) × (ℤ → α)) => AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival z.1,
tag_at_zero := AppliedModelingLib.Probability.Queueing.timedEmbeddedTaggedArrivalAtZero._proof_3 rate PtagPath,
arrivals_strict :=
AppliedModelingLib.Probability.Queueing.timedEmbeddedTaggedArrivalAtZero._proof_4 rate hrate PtagPath }
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival](#)
- [AppliedModelingLib.Probability.PoissonProcess.candidateTaggedArrivalAtZero](#)
- [AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure](#)
- [AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero](#)

AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkTaggedArrivalAtZero

Declaration location: reusable library

Lean declaration body

```
type:
{arrivalRate : ℝ} → 0 < arrivalRate → AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero ((ℤ → ℝ) × (ℤ → ℝ))

value:
fun {arrivalRate : ℝ} (harrivalRate : 0 < arrivalRate) =>
AppliedModelingLib.Probability.Queueing.timedEmbeddedTaggedArrivalAtZero arrivalRate harrivalRate
(AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure 1)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.twoSidedInterarrivalMeasure](#)
- [AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero](#)
- [AppliedModelingLib.Probability.Queueing.timedEmbeddedTaggedArrivalAtZero](#)

AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex

Declaration location: reusable library

Lean declaration body

```
type:
ℕ → Type

value:
fun (n : ℕ) => ( _ : Fin n) × ℤ
```

AppliedModelingLib.Queueing.StationaryPoissonWorkPath

Declaration location: reusable library

Lean declaration body

```
type:
Type

value:
AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)

AppliedModelingLib.Queueing.emptyNonpreemptivePriorityWorkState

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → {JobId : Type u_1} → ℝ → AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId
value:
fun {n : ℕ} {JobId : Type u_1} (time : ℝ) =>
{ currentTime := time, active := none, waiting := fun (x : Fin n) => [], completed := [] }
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)

AppliedModelingLib.Queueing.enqueueNonpreemptivePriorityJob

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
{JobId : Type u_1} →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId
value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId)
(job : AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId) =>
{ currentTime := state.currentTime, active := state.active,
waiting := Function.update state.waiting job.priority (state.waiting job.priority ++ [job]),
completed := state.completed }
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)

AppliedModelingLib.Queueing.finitePriorityInclusiveLoad

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ
value:
fun {n : ℕ} (meanService arrivalRate : Fin n → ℝ) (i : Fin n) =>
∑ j : Fin n, if j ≤ i then meanService j * arrivalRate j else 0
```

AppliedModelingLib.Queueing.finitePriorityInclusiveSlack

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ
value:
fun {n : ℕ} (meanService arrivalRate : Fin n → ℝ) (i : Fin n) =>
1 - AppliedModelingLib.Queueing.finitePriorityInclusiveLoad meanService arrivalRate i
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.finitePriorityInclusiveLoad](#)

AppliedModelingLib.Queueing.finitePriorityResidualWork

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → ℝ

value:
fun {n : ℕ} (arrivalRate meanService : Fin n → ℝ) => ∑ j : Fin n, arrivalRate j * meanService j ^ 2
```

AppliedModelingLib.Queueing.finitePriorityStrictLoad

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (meanService arrivalRate : Fin n → ℝ) (i : Fin n) =>
  ∑ j : Fin n, if j < i then meanService j * arrivalRate j else 0
```

AppliedModelingLib.Queueing.finitePriorityStrictSlack

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (meanService arrivalRate : Fin n → ℝ) (i : Fin n) =>
  1 - AppliedModelingLib.Queueing.finitePriorityStrictLoad meanService arrivalRate i
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.finitePriorityStrictLoad](#)

AppliedModelingLib.Queueing.finiteNonpreemptivePrioritySojournTime

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate meanService : Fin n → ℝ) (i : Fin n) =>
  AppliedModelingLib.Queueing.finiteNonpreemptivePriorityQueueWait arrivalRate meanService i + meanService i
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.finiteNonpreemptivePriorityQueueWait](#)

AppliedModelingLib.Queueing.hasPriorityWaitingJob

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → {JobId : Type u_1} → AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId → Prop

value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) =>
  ∃ (i : Fin n), 0 < (state.waiting i).length
```


Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)

AppliedModelingLib.Queueing.instDecidableEq_appliedModelingLib

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} → (a b : Class) → Decidable (a = b)

value:
fun {Class : Type u_1} (a b : Class) => Classical.decEq Class a b
```

AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedArrival

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_2} →
(i : Class) → AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i → Class →  $\mathbb{Z}$  →  $\mathbb{R}$ 

value:
fun {Class : Type u_2} (i : Class)
  (a : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i) (a_1 : Class)
  (a_2 :  $\mathbb{Z}$ ) =>
  if hji : a_1 = i then AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival a.1.1 a_2
  else AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkArrival (a.2 (a_1, hji)) a_2
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.candidatePalmArrival](#)
- [AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkArrival](#)
- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.instDecidableEq_appliedModelingLib](#)

AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedRequirement

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} →
(i : Class) →
(( $\mathbb{Z}$  →  $\mathbb{R}$ ) × ( $\mathbb{Z}$  →  $\mathbb{R}$ )) × ({j : Class // j ≠ i} → AppliedModelingLib.Queueing.StationaryPoissonWorkPath) →  $\mathbb{R}$ 

value:
fun {Class : Type u_1} (i : Class)
  (a : (( $\mathbb{Z}$  →  $\mathbb{R}$ ) × ( $\mathbb{Z}$  →  $\mathbb{R}$ )) × ({j : Class // j ≠ i} → AppliedModelingLib.Queueing.StationaryPoissonWorkPath)) =>
  a.1.2 0
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)

AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedRequirementAt

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_2} →
(i : Class) → AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i → Class →  $\mathbb{Z}$  →  $\mathbb{R}$ 

value:
fun {Class : Type u_2} (i : Class)
  (a : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i) (a_1 : Class)
  (a_2 :  $\mathbb{Z}$ ) =>
  if hji : a_1 = i then a.1.2 a_2 else (a.2 (a_1, hji)).2 a_2
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.instDecidableEq_appliedModelingLib](#)

AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkRestLaw

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} →
[Fintype Class] →
(arrivalRate : Class →  $\mathbb{R}$ ) →
(∀ (j : Class), 0 < arrivalRate j) →
(i : Class) →
  AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw
  ({ j : Class // j ≠ i } → AppliedModelingLib.Queueing.StationaryPoissonWorkPath)

value:
fun {Class : Type u_1} [Fintype Class] (arrivalRate : Class →  $\mathbb{R}$ ) (harrivalRate : ∀ (j : Class), 0 < arrivalRate j)
  (i : Class) =>
  AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkShiftInvariantLaw
  (fun (j : { j : Class // j ≠ i }) => arrivalRate ↑j)
  (AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkRestLaw._proof_1 arrivalRate harrivalRate i)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.Palm.ShiftInvariantProbabilityLaw](#)
- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkShiftInvariantLaw](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.instDecidableEq_appliedModelingLib](#)

AppliedModelingLib.Queueing.nonemptyPriorityClasses

Declaration location: reusable library

Lean declaration body

```
type:
{n :  $\mathbb{N}$ } → (Fin n →  $\mathbb{N}$ ) → Finset (Fin n)

value:
fun {n :  $\mathbb{N}$ } (backlog : Fin n →  $\mathbb{N}$ ) => {i : Fin n | 0 < backlog i}
```

AppliedModelingLib.Queueing.nextNonpreemptivePriority

Declaration location: reusable library

Lean declaration body

```
type:
{n :  $\mathbb{N}$ } → (backlog : Fin n →  $\mathbb{N}$ ) → (∃ (i : Fin n), 0 < backlog i) → Fin n

value:
fun {n :  $\mathbb{N}$ } (backlog : Fin n →  $\mathbb{N}$ ) (hnonempty : ∃ (i : Fin n), 0 < backlog i) =>
  (AppliedModelingLib.Queueing.nonemptyPriorityClasses backlog).min'
  (AppliedModelingLib.Queueing.nextNonpreemptivePriority._proof_1 backlog hnonempty)
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.nonemptyPriorityClasses](#)

AppliedModelingLib.Queueing.nextPriorityWaitingClass

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
{JobId : Type u_1} →
(state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) →
AppliedModelingLib.Queueing.hasPriorityWaitingJob state → Fin n

value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId)
(hwaiting : AppliedModelingLib.Queueing.hasPriorityWaitingJob state) =>
AppliedModelingLib.Queueing.nextNonpreemptivePriority (fun (i : Fin n) => (state.waiting i).length) hwaiting
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.hasPriorityWaitingJob](#)
- [AppliedModelingLib.Queueing.nextNonpreemptivePriority](#)

AppliedModelingLib.Queueing.nonpreemptivePriorityArrivalWindowIndices

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → (Fin n → Finset ℤ) → Finset (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n)

value:
fun {n : ℕ} (indices : Fin n → Finset ℤ) => Finset.univ.sigma indices
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)

AppliedModelingLib.Queueing.startNextNonpreemptivePriorityJob

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
{JobId : Type u_1} →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId

value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) =>
match state.active with
| some val => state
| none =>
if hwaiting : AppliedModelingLib.Queueing.hasPriorityWaitingJob state then
have i : Fin n := AppliedModelingLib.Queueing.nextPriorityWaitingClass state hwaiting;
match state.waiting i with
| [] => state
| job :: tail =>
{ currentTime := state.currentTime, active := some (job, job.serviceWork),
waiting := Function.update state.waiting i tail, completed := state.completed }
else state
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.hasPriorityWaitingJob](#)
- [AppliedModelingLib.Queueing.nextPriorityWaitingClass](#)

AppliedModelingLib.Queueing.completeNonpreemptivePriorityWorkJob

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
{JobId : Type u_1} →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId

value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) =>
  match state.active with
  | none => state
  | some (job, snd) =>
    AppliedModelingLib.Queueing.startNextNonpreemptivePriorityJob
    { currentTime := state.currentTime, active := none, waiting := state.waiting,
      completed := (job, state.currentTime) :: state.completed }
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.startNextNonpreemptivePriorityJob](#)

AppliedModelingLib.Queueing.advanceThenAdmitNonpreemptivePriorityJob

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
{JobId : Type u_1} →
  ℕ →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
  AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId

value:
fun {n : ℕ} {JobId : Type u_1} (fuel : ℕ) (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId)
  (job : AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId) =>
  AppliedModelingLib.Queueing.admitNonpreemptivePriorityJob
  (AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState fuel job.arrivalTime state) job
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.admitNonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState](#)

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} →
(i : Class) → AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i → Class → ℤ → ℝ

value:
fun {Class : Type u_1} (i : Class)
  (a : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i) (a_1 : Class)
  (a_2 : ℤ) =>
  AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedArrival i a a_1 a_2
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedArrival](#)

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexKey

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
  AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n → Lex (ℝ × Σ i (j : Fin n), ℤ)

value:
fun {n : ℕ} (i : Fin n) (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
  (q : AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) =>
  toLex (AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival i z q.fst q.snd, toLex q)
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival](#)

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexLE

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
  AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n →
  AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n → Prop

value:
fun {n : ℕ} (i : Fin n) (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
  (first second : AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) =>
  AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexKey i z first ≤
  AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexKey i z second
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexKey](#)

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFutureArrivalIndices

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i → ℝ → Fin n → Finset ℤ

value:
fun {n : ℕ} (i : Fin n) (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
  (t : ℝ) (a : Fin n) =>
  if hji : a = i then AppliedModelingLib.Probability.PoissonProcess.palmTaggedArrivalIndicesRightClosed 0 t z.1.1
  else AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrivalIndicesRightClosed 0 t (z.2 (a, hji)).1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.palmTaggedArrivalIndicesRightClosed](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrivalIndicesRightClosed](#)
- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)

AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFutureArrivalIndices

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
  ℝ → List (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n)

value:
fun {n : ℕ} (i : Fin n) (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
  (t : ℝ) =>
  (AppliedModelingLib.Queueing.nonpreemptivePriorityArrivalWindowIndices
    (AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFutureArrivalIndices i z t)).sort
  (AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexLE i z)
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.nonpreemptivePriorityArrivalWindowIndices](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexLE](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFutureArrivalIndices](#)

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirementAt

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} →
(Class → ℝ) →
(i : Class) → AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i → Class → ℤ → ℝ

value:
fun {Class : Type u_1} (meanService : Class → ℝ) (i : Class)
  (a : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i) (a_1 : Class)
  (a_2 : ℤ) =>
  meanService a_1 * AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedRequirementAt i a a_1 a_2
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedRequirementAt](#)

AppliedModelingLib.Queueing.stationaryPriorityInputDecidableEq

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} → (a b : Class) → Decidable (a = b)

value:
fun {Class : Type u_1} (a b : Class) => Classical.decEq Class a b
```

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalWindowIndices

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} →
(i : Class) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i → ℝ → ℝ → Class → Finset ℤ

value:
fun {Class : Type u_1} (i : Class)
  (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample Class i) (a b : ℝ) (j : Class) =>
  if hji : j = i then AppliedModelingLib.Probability.PoissonProcess.palmTaggedArrivalIndices a b z.1.1
  else AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrivalIndices a b (z.2 (j, hji)).1
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.palmTaggedArrivalIndices](#)
- [AppliedModelingLib.Probability.PoissonProcess.suspensionBaseArrivalIndices](#)
- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.stationaryPriorityInputDecidableEq](#)

AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedArrivalWindowIndices

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
  ℝ → ℝ → List (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n)

value:
fun {n : ℕ} (i : Fin n) (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
  (a b : ℝ) =>
  (AppliedModelingLib.Queueing.nonpreemptivePriorityArrivalWindowIndices
    (AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalWindowIndices i z a b)).sort
  (AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexLE i z)
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.nonpreemptivePriorityArrivalWindowIndices](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalIndexLE](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrivalWindowIndices](#)

AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedArrivalWindowJobs

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} →
(Fin n → ℝ) →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
  ℝ →
  ℝ →
  List
    (AppliedModelingLib.Queueing.NonpreemptivePriorityJob n
      (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n))

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
  (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) (a b : ℝ) =>
  List.map
    (fun (q : AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) =>
      { identifier := q, priority := q.fst,
        arrivalTime := AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival i z q.fst q.snd,
        serviceWork :=
          AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirementAt meanService i z q.fst q.snd })
    (AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedArrivalWindowIndices i z a b)
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedArrivalWindowIndices](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirementAt](#)

AppliedModelingLib.Queueing.totalPriorityWaitingJobs

Declaration location: reusable library

Lean declaration body

```
type:
{n : ℕ} → {JobId : Type u_1} → AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId → ℕ
value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) =>
  ∑ i : Fin n, (state.waiting i).length
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)

AppliedModelingLib.finiteDelayNetValue

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} → [Fintype Class] → (Class → ℝ → ℝ) → (Class → ℝ) → (Class → (Class → ℝ) → ℝ) → (Class → ℝ) → ℝ
value:
fun {Class : Type u_1} [Fintype Class] (value : Class → ℝ → ℝ) (delayCost : Class → ℝ)
  (waitingTime : Class → (Class → ℝ) → ℝ) (x : Class → ℝ) =>
  ∑ j : Class, (value j (x j) - delayCost j * x j * waitingTime j x)
```

AppliedModelingLib.finiteMarginalExternalityPrice

Declaration location: reusable library

Lean declaration body

```
type:
{Class : Type u_1} → [Fintype Class] → (Class → ℝ) → (Class → ℝ) → (Class → ℝ) → ℝ
value:
fun {Class : Type u_1} [Fintype Class] (delayCost flow partialWaiting : Class → ℝ) =>
  ∑ j : Class, delayCost j * flow j * partialWaiting j
```

MendelsonWhang1990PriorityPricing.externalityPrice

Declaration location: paper-local

Lean declaration body

```
type:
{Class : Type u_1} → [Fintype Class] → (Class → ℝ) → (Class → ℝ) → (Class → ℝ) → ℝ
value:
fun {Class : Type u_1} [Fintype Class] (delayCost flow partialWaiting : Class → ℝ) =>
  AppliedModelingLib.finiteMarginalExternalityPrice delayCost flow partialWaiting
```

Direct retained dependencies

- [AppliedModelingLib.finiteMarginalExternalityPrice](#)

MendelsonWhang1990PriorityPricing.paperInterfaceClassicalDecidableEq

Declaration location: paper-local

Lean declaration body

```
type:
(α : Type u_1) → (a b : α) → Decidable (a = b)
value:
fun (α : Type u_1) (a b : α) => Classical.decEq α a b
```


MendelsonWhang1990PriorityPricing.priorityLinearNumerator

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate delayCost meanService : Fin n → ℝ) (i : Fin n) =>
  delayCost i * arrivalRate i * AppliedModelingLib.Queueing.finitePriorityResidualWork arrivalRate meanService
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.finitePriorityResidualWork](#)

MendelsonWhang1990PriorityPricing.prioritySojournDerivative

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate meanService : Fin n → ℝ) (arrivalClass priorityClass : Fin n) =>
  (meanService arrivalClass ^ 2 *
    (AppliedModelingLib.Queueing.finitePriorityStrictSlack meanService arrivalRate priorityClass *
     AppliedModelingLib.Queueing.finitePriorityInclusiveSlack meanService arrivalRate priorityClass) -
    AppliedModelingLib.Queueing.finitePriorityResidualWork arrivalRate meanService *
    ((-if arrivalClass < priorityClass then meanService arrivalClass else 0) *
     AppliedModelingLib.Queueing.finitePriorityInclusiveSlack meanService arrivalRate priorityClass +
     AppliedModelingLib.Queueing.finitePriorityStrictSlack meanService arrivalRate priorityClass *
     -if arrivalClass ≤ priorityClass then meanService arrivalClass else 0)) /
    (AppliedModelingLib.Queueing.finitePriorityStrictSlack meanService arrivalRate priorityClass *
     AppliedModelingLib.Queueing.finitePriorityInclusiveSlack meanService arrivalRate priorityClass) ^
  2
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.finitePriorityInclusiveSlack](#)
- [AppliedModelingLib.Queueing.finitePriorityResidualWork](#)
- [AppliedModelingLib.Queueing.finitePriorityStrictSlack](#)

MendelsonWhang1990PriorityPricing.prioritySojournTime

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate meanService : Fin n → ℝ) (i : Fin n) =>
  AppliedModelingLib.Queueing.finiteNonpreemptivePrioritySojournTime arrivalRate meanService i
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.finiteNonpreemptivePrioritySojournTime](#)

MendelsonWhang1990PriorityPricing.priorityPopulation

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate meanService : Fin n → ℝ) (i : Fin n) =>
  arrivalRate i * MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate meanService i
```

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.prioritySojournTime](#)

MendelsonWhang1990PriorityPricing.priorityScheduleSojournTime

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → Equiv.Perm (Fin n) → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate meanService : Fin n → ℝ) (schedule : Equiv.Perm (Fin n)) (customer : Fin n) =>
  MendelsonWhang1990PriorityPricing.prioritySojournTime (arrivalRate ∘ ↑ schedule) (meanService ∘ ↑ schedule)
  ((Equiv.symm schedule) customer)
```

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.prioritySojournTime](#)

MendelsonWhang1990PriorityPricing.priorityScheduleDelayCost

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → (Fin n → ℝ) → Equiv.Perm (Fin n) → ℝ

value:
fun {n : ℕ} (arrivalRate delayCost meanService : Fin n → ℝ) (schedule : Equiv.Perm (Fin n)) =>
  ∑ customer : Fin n,
    delayCost customer * arrivalRate customer *
    MendelsonWhang1990PriorityPricing.priorityScheduleSojournTime arrivalRate meanService schedule customer
```

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.priorityScheduleSojournTime](#)

MendelsonWhang1990PriorityPricing.priorityScheduleNetValue

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ → ℝ) → (Fin n → ℝ) → (Fin n → ℝ) → (Fin n → ℝ) → Equiv.Perm (Fin n) → ℝ

value:
fun {n : ℕ} (value : Fin n → ℝ → ℝ) (arrivalRate delayCost meanService : Fin n → ℝ) (schedule : Equiv.Perm (Fin n)) =>
  ∑ customer : Fin n, value customer (arrivalRate customer) -
    MendelsonWhang1990PriorityPricing.priorityScheduleDelayCost arrivalRate delayCost meanService schedule
```

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.priorityScheduleDelayCost](#)

MendelsonWhang1990PriorityPricing.prioritySuccessorOrZero

Declaration location: paper-local

Lean declaration body

```
type:
{n : ℕ} → (Fin n → ℝ) → Fin n → ℝ

value:
fun {n : ℕ} (value : Fin n → ℝ) (i : Fin n) => if h : ↑i + 1 < n then value (↑i + 1, h) else 0
```

Material library prerequisites

AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkMeasure

Source locator: cited publication, lines 227-242

Verbatim paper-source connection

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

A1. Arrival Processes

Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate λ_i . The R arrival processes are stochastically independent.

by the system (per unit of time) to the organization, where we assume for now that the system administra-

[Next verbatim source excerpt]

L_q^* = The mean number of class- i jobs in the queue in steady state;
 $W_i(\cdot)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i(\cdot)$ = The expected sojourn time of a class- i job in the system in steady state.

KLEINROCK, L. 1967. Optimum Bribing for Queue Position. *Opns. Res.* 15, 304-318.
KLEINROCK, L. 1976. *Queueing System, Vols. I and II.* John Wiley, New York.
KNUDSEN, N. C. 1972. Individual and Social Optimization in a Multi-Server Queue With a General Cost-

Lean-expanded library semantic target

```
type:
{Class : Type u_1} →
[FIntype Class] →
(Class → ℝ) →
MeasureTheory.Measure (Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ))
```

```
value:
fun {Class : Type u_1} [FIntype Class] (arrivalRate : Class → ℝ) =>
MeasureTheory.Measure.pi fun (i : Class) =>
AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkMeasure (arrivalRate i)
```

Exact Lean library declaration

```
-- One independently sampled stationary marked-Poisson input per class.
Each coordinate contains both its stationary arrival suspension and a complete
two-sided iid unit-exponential work-mark path. -/
noncomputable def multiclassStationaryPoissonWorkMeasure (arrivalRate : Class -> Real) :
Measure (Class -> (GoodSuspensionState × (Int -> Real))) :=
Measure.pi fun i => Queueing.stationaryPoissonWorkMeasure (arrivalRate i)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkMeasure](#)

Recorded source-to-library screening Verdict: matches

Reason: A1's independent stationary class inputs are represented by the product stationary marked-Poisson work measure; work marks are the service-process carrier, not an added claim.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Probability.stabilizedFiniteReplayResponse

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(\cdot)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i(\cdot)$ = The expected sojourn time of a class- i job in the system in steady state.

For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

$$W_i(L1) = W_1 W_{q1} + \frac{C_i A R}{c_j} (A - c_j) \quad 1131-1140.$$

[Next verbatim source excerpt]

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(\cdot)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i(5)$
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay payment of $v_i * W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

Al. Arrival Processes
Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.

by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded library semantic target

type:
 $\{\Omega : \text{Type } u_1\} \rightarrow (\mathbb{N} \rightarrow \Omega \rightarrow \mathbb{R}) \rightarrow \Omega \rightarrow \mathbb{R}$

value:
 $\text{fun } \{\Omega : \text{Type } u_1\} (\text{finiteResponse} : \mathbb{N} \rightarrow \Omega \rightarrow \mathbb{R}) (a : \Omega) =>$
 $\text{Filter.limsup } (\text{fun } (n : \mathbb{N}) => \text{finiteResponse } n \ a) \ \text{Filter.atTop}$

Exact Lean library declaration

-- The canonical response induced by a sequence of total finite replay responses. Under eventual equality it is the eventual literal value. -/
noncomputable def stabilizedFiniteReplayResponse
(finiteResponse : Nat → Ω → ℝ) : Ω → ℝ :=
fun ω => limsup (fun n => finiteResponse n ω) atTop

Recorded source-to-library screening Verdict: matches

Reason: The expanded value takes the limsup of total finite replay response observations. On the proved eventual-stabilization event this equals the literal stationary tagged response, so it is a total realization of the steady-state response quantity used in Appendix (A-1), not a replacement queue formula.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source locator: cited publication, lines 227-242

Verbatim paper-source connection

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

A1. Arrival Processes

Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI1 Queue / 873

dependent arrival rate λ_i . The R arrival processes are stochastically independent.

by the system (per unit of time) to the organization, where we assume for now that the system administra-

[Next verbatim source excerpt]

$L_q^*)$ = The mean number of class- i jobs in the queue in steady state;
 $W_i(\cdot)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i(\cdot)$ = The expected sojourn time of a class- i job in the system in steady state.

KLEINROCK, L. 1967. Optimum Bribing for Queue Position. *Opns. Res.* 15, 304-318.
KLEINROCK, L. 1976. *Queueing System, Vols. I and II.* John Wiley, New York.
KNUDSEN, N. C. 1972. Individual and Social Optimization in a Multi-Server Queue With a General Cost-

Lean-expanded library semantic target

```
type:
(Class : Type u_2) → Class → Type (max 0 u_2)

value:
fun (Class : Type u_2) (i : Class) =>
  ((ℤ → ℝ) × (ℤ → ℝ)) × {j : Class // j ≠ i} → AppliedModelingLib.Queueing.StationaryPoissonWorkPath)
```

Exact Lean library declaration

```
-- The complete input carrier seen from one selected class arrival. The
selected class uses its Palm-recentered arrival and mark paths, while every
other class retains a shifted stationary marked-Poisson path. -/
abbrev MulticlassStationaryPoissonWorkClassTaggedSample
  (Class : Type*) (i : Class) :=
  ((Int → Real) × (Int → Real)) ×
  {j : Class // j ≠ i} → StationaryPoissonWorkPath)
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)

Recorded source-to-library screening Verdict: matches

Reason: This is the A1 selected-arrival/Palm carrier: the selected class is recentered at its arrival and other classes remain stationary.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.NonpreemptivePriorityJob

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process $V_i'(X_i) = \pi_i + v_i W_i$ (5)
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite cost, given by $v_i * W_i(X)$.
mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean declaration type (metadata)

field identifier:
 $\{n : \mathbb{N}\} \rightarrow \{\text{JobId} : \text{Type } u_1\} \rightarrow \text{AppliedModelingLib.Queueing.NonpreemptivePriorityJob } n \text{ JobId} \rightarrow \text{JobId}$
field priority:
 $\{n : \mathbb{N}\} \rightarrow \{\text{JobId} : \text{Type } u_1\} \rightarrow \text{AppliedModelingLib.Queueing.NonpreemptivePriorityJob } n \text{ JobId} \rightarrow \text{Fin } n$
field arrivalTime:
 $\{n : \mathbb{N}\} \rightarrow \{\text{JobId} : \text{Type } u_1\} \rightarrow \text{AppliedModelingLib.Queueing.NonpreemptivePriorityJob } n \text{ JobId} \rightarrow \mathbb{R}$
field serviceWork:
 $\{n : \mathbb{N}\} \rightarrow \{\text{JobId} : \text{Type } u_1\} \rightarrow \text{AppliedModelingLib.Queueing.NonpreemptivePriorityJob } n \text{ JobId} \rightarrow \mathbb{R}$

Exact Lean library declaration

```
-- A finite-trace customer, retaining its identifier, declared class,
physical arrival time, and required service work. -/
structure NonpreemptivePriorityJob (n : ℕ) (JobId : Type*) where
  identifier : JobId
  priority : Fin n
  arrivalTime : ℝ
  serviceWork : ℝ
```

Recorded source-to-library screening Verdict: matches

Reason: The expanded record retains exactly the data of one A4 queue customer needed by the nonpreemptive-priority dynamics: identity, class priority, arrival epoch, and service requirement. It introduces no scheduling behavior by itself.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i(5)$
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay given by $v_i * W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean declaration type (metadata)

```
field currentTime:
{n : ℕ} → {JobId : Type u_1} → AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId → ℝ

field active:
{n : ℕ} →
{JobId : Type u_1} →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
Option (AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId × ℝ)

field waiting:
{n : ℕ} →
{JobId : Type u_1} →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
Fin n → List (AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId)

field completed:
{n : ℕ} →
{JobId : Type u_1} →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
List (AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId × ℝ)
```

Exact Lean library declaration

```
-- A work-level queue state. The active pair stores the currently served
job and its residual work; `waiting` consists of one FIFO list per class; and
`completed` records each completed job with its physical completion time. -/
structure NonpreemptivePriorityWorkState (n : ℕ) (JobId : Type*) where
  currentTime : ℝ
  active : Option (NonpreemptivePriorityJob n JobId × ℝ)
  waiting : Fin n → List (NonpreemptivePriorityJob n JobId)
  completed : List (NonpreemptivePriorityJob n JobId × ℝ)
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)

Recorded source-to-library screening Verdict: matches

Reason: The state records A4's one-server active/residual-work and class-FIFO nonpreemptive-priority queue discipline.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.admitNonpreemptivePriorityJob

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process $V_i'(X_i) = \pi_i + v_i W_i$ (5)
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite cost, given by $v_i * W_i(X)$.
mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
{JobId : Type u_1} →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
  AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId

value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId)
  (job : AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId) =>
  have prepared : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId :=
    AppliedModelingLib.Queueing.startNextNonpreemptivePriorityJob state;
  match prepared.active with
  | none =>
    { currentTime := prepared.currentTime, active := some (job, job.serviceWork), waiting := prepared.waiting,
      completed := prepared.completed }
  | some val => AppliedModelingLib.Queueing.enqueueNonpreemptivePriorityJob prepared job
```

Exact Lean library declaration

```
-- An arrival joins service immediately only when the server is idle after
starting any previously waiting work; otherwise it joins the tail of its own
class-FIFO list. -/
noncomputable def admitNonpreemptivePriorityJob
  {n : ℕ} {JobId : Type*}
  (state : NonpreemptivePriorityWorkState n JobId)
  (job : NonpreemptivePriorityJob n JobId) :
  NonpreemptivePriorityWorkState n JobId :=
by
  classical
  let prepared := startNextNonpreemptivePriorityJob state
  exact match prepared.active with
  | none => { prepared with active := some (job, job.serviceWork) }
  | some _ => enqueueNonpreemptivePriorityJob prepared job
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.enqueueNonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.startNextNonpreemptivePriorityJob](#)

Recorded source-to-library screening Verdict: matches

Reason: This is A4's nonpreemptive admission and within-class FIFO enqueue operation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(\cdot)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i(X_i)$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay payment of $v_i * W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
{JobId : Type u_1} →
ℕ →
ℝ →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId

value:
fun {n : ℕ} {JobId : Type u_1} (a : ℕ) (a_1 : ℝ)
(a_2 : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) =>
Nat.brecOn (motive := fun (x : ℕ) =>
ℝ →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId)
a AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState._f a_1 a_2
```

Exact Lean library declaration

```
-- Advance a finite queue state to a target time, processing at most `fuel`
service completions. With enough fuel to cover the currently resident jobs,
this is the literal nonpreemptive service evolution up to that time. -/
noncomputable def advanceNonpreemptivePriorityWorkState
{n : ℕ} {JobId : Type*} :
ℕ → ℝ → NonpreemptivePriorityWorkState n JobId →
NonpreemptivePriorityWorkState n JobId
| 0, target, state =>
if htarget : target ≤ state.currentTime then state
else match state.active with
| none => { state with currentTime := target }
| some _ => state
| fuel + 1, target, state =>
if htarget : target ≤ state.currentTime then state
else match state.active with
| none => { state with currentTime := target }
| some (job, residual) =>
if hcomplete : residual ≤ target - state.currentTime then
advanceNonpreemptivePriorityWorkState fuel target
(completeNonpreemptivePriorityWorkJob
{ state with currentTime := state.currentTime + residual })
else
{ state with
currentTime := target
active := some (job, residual - (target - state.currentTime)) }
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.completeNonpreemptivePriorityWorkJob](#)

Recorded source-to-library screening Verdict: matches

Reason: This is A4's work-state evolution through a completion before selecting the next priority job.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(\cdot)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{n : ℕ} → {JobId : Type u_1} → AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId → ℕ

value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) =>
  (match state.active with
  | none => 0
  | some val => 1) +
  AppliedModelingLib.Queueing.totalPriorityWaitingJobs state
```

Exact Lean library declaration

```
-- The total number of jobs currently resident in the queue, including an
active job if one exists. -/
def totalNonpreemptivePriorityWorkJobs
  {n : ℕ} {JobId : Type*}
  (state : NonpreemptivePriorityWorkState n JobId) : ℕ :=
  (match state.active with | none => 0 | some _ => 1) +
  totalPriorityWaitingJobs state
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.totalPriorityWaitingJobs](#)

Recorded source-to-library screening Verdict: matches

Reason: This finite fuel/count helper supports A4 execution without changing the source queue discipline.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.runNonpreemptivePriorityArrivalTrace

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i(X_i)$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay given by $v_i * W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
{JobId : Type u_1} →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
  List (AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId) →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId

value:
fun {n : ℕ} {JobId : Type u_1} (initial : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId)
  (jobs : List (AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId)) =>
  List.foldl
    (fun (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId)
      (job : AppliedModelingLib.Queueing.NonpreemptivePriorityJob n JobId) =>
        AppliedModelingLib.Queueing.advanceThenAdmitNonpreemptivePriorityJob
          (AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs state) state job)
    initial jobs
```

Exact Lean library declaration

```
-- Execute a finite, chronologically supplied list of arrivals. The list
-- order is the tie convention if two jobs are presented at the same epoch. -/
noncomputable def runNonpreemptivePriorityArrivalTrace
  {n : ℕ} {JobId : Type*}
  (initial : NonpreemptivePriorityWorkState n JobId)
  (jobs : List (NonpreemptivePriorityJob n JobId)) :
  NonpreemptivePriorityWorkState n JobId :=
  jobs.foldl (fun state job =>
    advanceThenAdmitNonpreemptivePriorityJob
      (totalNonpreemptivePriorityWorkJobs state) state job) initial
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.advanceThenAdmitNonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs](#)

Recorded source-to-library screening Verdict: matches

Reason: It is the chronological finite execution of A4's nonpreemptive priority discipline.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
 The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
 where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay given by $v_i * W_i(X)$.
 The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```

type:
{n : ℕ} →
  (Fin n → ℝ) →
    (i : Fin n) →
      AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
        ℝ →
          AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
            (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n)

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
  (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) (a b : ℝ) =>
  have initial :
    AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
      (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) :=
    AppliedModelingLib.Queueing.emptyNonpreemptivePriorityWorkState a;
  have afterArrivals :
    AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
      (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) :=
    AppliedModelingLib.Queueing.runNonpreemptivePriorityArrivalTrace initial
      (AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedArrivalWindowJobs meanService i z a b);
  AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState
    (AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs afterArrivals) b afterArrivals

```

Exact Lean library declaration

```

/-- Execute the canonically ordered finite selected/Palm arrival ledger,
starting empty at the left endpoint and serving through the right endpoint. -/
def canonicalStationaryPriorityClassTaggedFiniteWindowState
  {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
  (z : MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
  (a b : ℝ) :
    NonpreemptivePriorityWorkState n (NonpreemptivePriorityArrivalIndex n) :=
  let initial := emptyNonpreemptivePriorityWorkState
    (n := n) (jobId := NonpreemptivePriorityArrivalIndex n) a
  let afterArrivals := runNonpreemptivePriorityArrivalTrace initial
    (canonicalStationaryPriorityClassTaggedArrivalWindowJobs meanService i z a b)
  advanceNonpreemptivePriorityWorkState
    (totalNonpreemptivePriorityWorkJobs afterArrivals) b afterArrivals

```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedArrivalWindowJobs](#)
- [AppliedModelingLib.Queueing.emptyNonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.runNonpreemptivePriorityArrivalTrace](#)
- [AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs](#)

Recorded source-to-library screening Verdict: matches

Reason: This finite replay applies the A4 discipline to the separately mapped A1 stationary input, without being promoted to a source formula itself.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFutureArrivalJobs

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i(5)$
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay given by $v_i * W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
(Fin n → ℝ) →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
    ℝ →
      List
        (AppliedModelingLib.Queueing.NonpreemptivePriorityJob n
         (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n))

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
  (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) (t : ℝ) =>
  List.map
    (fun (q : AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) =>
      { identifier := q, priority := q.fst,
        arrivalTime := AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival i z q.fst q.snd,
        serviceWork :=
          AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirementAt meanService i z q.fst q.snd })
    (AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFutureArrivalIndices i z t)
```

Exact Lean library declaration

```
-- The literal finite list of post-tag priority jobs used to continue the
post-admission selected queue state. -/
noncomputable def canonicalStationaryPriorityClassTaggedFutureArrivalJobs
  {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
  (z : MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
  (t : ℝ) : List (NonpreemptivePriorityJob n (NonpreemptivePriorityArrivalIndex n)) :=
  (canonicalStationaryPriorityClassTaggedFutureArrivalIndices i z t).map fun q =>
  { identifier := q
  priority := q.1
  arrivalTime := stationaryPriorityClassTaggedArrival i z q.1 q.2
  serviceWork := stationaryPriorityClassTaggedWorkRequirementAt meanService i z q.1 q.2 }
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFutureArrivalIndices](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirementAt](#)

Recorded source-to-library screening Verdict: matches

Reason: It converts the mapped post-tag A1 input to jobs for the A4 priority queue, preserving class and work marks.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.clearNonpreemptivePriorityCompletionLedger

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process $V_i'(X_i) = \pi_i + v_i W_i$ (5)
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite cost, given by $v_i * W_i(X)$.
mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
{JobId : Type u_1} →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId

value:
fun {n : ℕ} {JobId : Type u_1} (state : AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n JobId) =>
{ currentTime := state.currentTime, active := state.active, waiting := state.waiting, completed := [] }
```

Exact Lean library declaration

```
/- Reset the observational completion ledger while retaining the complete live priority queue. This does not alter any future service or admission decision, and is useful when a new tagged observation begins at a specified physical epoch. -/
def clearNonpreemptivePriorityCompletionLedger
  {n : ℕ} {JobId : Type*}
  (state : NonpreemptivePriorityWorkState n JobId) :
  NonpreemptivePriorityWorkState n JobId :=
{ state with completed := [] }
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)

Recorded source-to-library screening Verdict: matches

Reason: This observation-only ledger reset preserves the A4 live queue dynamics while beginning a tagged completion observation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.finiteNonpreemptivePriorityQueueWait

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(.)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i()$ = The expected sojourn time of a class- i job in the system in steady state.

For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

$$W_i(X) = W_1 W_{q1} + \frac{C_i A_i}{c_j} (A_i - c_j) \quad 1131-1140.$$

KLEINROCK, L. 1976. Queueing System, Vols. I and II. John Wiley, New York.
KNUDSEN, N. C. 1972. Individual and Social Optimization in a Multi-Server Queue With a General Cost-Benefit Structure. *Econometrica* 40, 515-528.
KREPS, D. 1990. A Course in Microeconomic Theory. Princeton University Press, Princeton, N.J.
MARCHAND, M. 1974. Priority Pricing. *Mgmt. Sci.* 20,

[Next verbatim source excerpt]

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = p_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of p_i per job, and the expected class- i delay payment of $v_i * W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

A1. Arrival Processes
Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and the arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent. by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded library semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:
fun $\{n : \mathbb{N}\}$ (arrivalRate meanService : $\text{Fin } n \rightarrow \mathbb{R}$) (i : $\text{Fin } n$) =>
AppliedModelingLib.Queueing.finitePriorityResidualWork arrivalRate meanService /
(AppliedModelingLib.Queueing.finitePriorityStrictSlack meanService arrivalRate i *
AppliedModelingLib.Queueing.finitePriorityInclusiveSlack meanService arrivalRate i)

Exact Lean library declaration

/-- Standard nonpreemptive-priority mean waiting time in queue, expressed in terms of the residual-work numerator and the two priority-load slacks. -/
noncomputable def finiteNonpreemptivePriorityQueueWait
{n : ℕ} (arrivalRate meanService : Fin n → ℝ) (i : Fin n) : ℝ :=
finitePriorityResidualWork arrivalRate meanService /
(finitePriorityStrictSlack meanService arrivalRate i *
finitePriorityInclusiveSlack meanService arrivalRate i)

Direct retained dependencies

- [AppliedModelingLib.Queueing.finitePriorityInclusiveSlack](#)
- [AppliedModelingLib.Queueing.finitePriorityResidualWork](#)
- [AppliedModelingLib.Queueing.finitePriorityStrictSlack](#)

Recorded source-to-library screening Verdict: matches

Reason: Its residual-work numerator and strict/inclusive priority-load slacks are exactly the Appendix (A-1) mean queue-wait ingredients.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

AppliedModelingLib.Queueing.nonpreemptivePriorityRecordedCompletionTime

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(.)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i()$ = The expected sojourn time of a class- i job in the system in steady state.

For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

$$W_i(L1) = W_1 W_{q1} + \frac{C_i A_i}{c_j} (A_i - 1) \quad 1131-1140.$$

KLEINROCK, L. 1976. Queueing System, Vols. I and II. John Wiley, New York.
KNUDSEN, N. C. 1972. Individual and Social Optimization in a Multi-Server Queue With a General Cost-Benefit Structure. *Econometrica* 40, 515-528.
KREPS, D. 1990. A Course in Microeconomic Theory. Princeton University Press, Princeton, N.J.
MARCHAND, M. 1974. Priority Pricing. *Mgmt. Sci.* 20,

[Next verbatim source excerpt]

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay payment of $v_i * W_i(X)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

Al. Arrival Processes
Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and
The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.
by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded library semantic target

```
type:
{n : ℕ} →
  AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
  (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) →
  AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n → Option ℝ

value:
fun {n : ℕ}
  (state :
    AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
    (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n))
  (identifier : AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) =>
  Option.map Prod.snd
  (List.find?
    (fun
      (entry :
        AppliedModelingLib.Queueing.NonpreemptivePriorityJob n
        (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) ×
        ℝ) =>
        decide (entry.1.identifier = identifier))
    state.completed.reverse)
```

Exact Lean library declaration

```
-- The first physical completion epoch recorded for a fixed labelled job in
a finite queue state. Completion records are prepended as service occurs, so
the ledger is read in chronological order to keep this observation unchanged
by later, unrelated continuations. -/
noncomputable def nonpreemptivePriorityRecordedCompletionTime
  {n : ℕ}
  (state : NonpreemptivePriorityWorkState n (NonpreemptivePriorityArrivalIndex n))
  (identifier : NonpreemptivePriorityArrivalIndex n) : Option ℝ :=
  (state.completed.reverse.find? fun entry => decide (entry.1.identifier = identifier)).map Prod.snd
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)

- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)

Recorded source-to-library screening **Verdict:** matches

Reason: This finite replay completion observation supports the Appendix tagged queue-wait quantity without asserting a separate source theorem.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedJob

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{n : ℕ} →
(Fin n → ℝ) →
(i : Fin n) →
  AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
  AppliedModelingLib.Queueing.NonpreemptivePriorityJob n
  (AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n)

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
(z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) =>
{ identifier := (i, 0), priority := i,
  arrivalTime := AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival i z i 0,
  serviceWork := AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirementAt meanService i z i 0 }
```

Exact Lean library declaration

```
-- The distinguished class-`i` customer at the Palm origin, represented as
an ordinary priority job with its original class/index identifier. -/
def stationaryPriorityClassTaggedJob
{n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
(z : MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) :
NonpreemptivePriorityJob n (NonpreemptivePriorityArrivalIndex n) :=
{ identifier := Sigma.mk i 0
  priority := i
  arrivalTime := stationaryPriorityClassTaggedArrival i z i 0
  serviceWork := stationaryPriorityClassTaggedWorkRequirementAt meanService i z i 0 }
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedArrival](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirementAt](#)

Recorded source-to-library screening Verdict: matches

Reason: It represents the selected class customer at the Palm origin as an ordinary A4 priority job.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(.)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i()$ = The expected sojourn time of a class- i job in the system in steady state.
 For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

$$W_i(L1) = W_1 W_{q1} + \frac{C_i A_i}{c_j} (A_i - 1) \quad 1131-1140.$$

[Next verbatim source excerpt]

A4. Service Process
 The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$$V_i'(X_i) = \pi_i + v_i W_i \quad (5)$$
 where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $W_i(X_i)$. The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$. **A1. Arrival Processes**
 Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and
 The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
 108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
 All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.
 by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded library semantic target

```

type:
{n : ℕ} →
(Fin n → ℝ) →
(i : Fin n) →
AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i →
ℝ →
ℝ →
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
(AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n)

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
(z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) (older t : ℝ) =>
have preArrival :
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
(AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) :=
AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFiniteWindowState meanService i z (-older) 0;
have afterArrivals :
AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState n
(AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex n) :=
AppliedModelingLib.Queueing.runNonpreemptivePriorityArrivalTrace
(AppliedModelingLib.Queueing.admitNonpreemptivePriorityJob
(AppliedModelingLib.Queueing.clearNonpreemptivePriorityCompletionLedger preArrival)
(AppliedModelingLib.Queueing.stationaryPriorityClassTaggedJob meanService i z))
(AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFutureArrivalJobs meanService i z t);
AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState
(AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs afterArrivals) t afterArrivals

```

Exact Lean library declaration

```

/-- A completely finite two-sided replay: execute the selected/Palm ledger
from a finite past horizon to zero, clear its historical observations, admit
the distinguished customer, and continue through a finite future horizon. -/
noncomputable def stationaryPriorityClassTaggedFiniteReplayPostArrivalState
{n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
(z : MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
(older t : ℝ) : NonpreemptivePriorityWorkState n (NonpreemptivePriorityArrivalIndex n) :=
let preArrival := canonicalStationaryPriorityClassTaggedFiniteWindowState
meanService i z (-older) 0
let afterArrivals := runNonpreemptivePriorityArrivalTrace

```

```
(admitNonpreemptivePriorityJob
 (clearNonpreemptivePriorityCompletionLedger preArrival)
 (stationaryPriorityClassTaggedJob meanService i z))
 (canonicalStationaryPriorityClassTaggedFutureArrivalJobs meanService i z t)
 advanceNonpreemptivePriorityWorkState
 (totalNonpreemptivePriorityWorkJobs afterArrivals) t afterArrivals
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityArrivalIndex](#)
- [AppliedModelingLib.Queueing.NonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.admitNonpreemptivePriorityJob](#)
- [AppliedModelingLib.Queueing.advanceNonpreemptivePriorityWorkState](#)
- [AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFiniteWindowState](#)
- [AppliedModelingLib.Queueing.canonicalStationaryPriorityClassTaggedFutureArrivalJobs](#)
- [AppliedModelingLib.Queueing.clearNonpreemptivePriorityCompletionLedger](#)
- [AppliedModelingLib.Queueing.runNonpreemptivePriorityArrivalTrace](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedJob](#)
- [AppliedModelingLib.Queueing.totalNonpreemptivePriorityWorkJobs](#)

Recorded source-to-library screening Verdict: matches

Reason: It constructs the finite two-sided A1/A4 replay used to support the stationary tagged wait calculation.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(.)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i()$ = The expected sojourn time of a class- i job in the system in steady state.
 For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

$$W_i(L1) = W_i W_{q1} + \frac{C_i A_i}{c_j} (A_i - 1) \quad 1131-1140.$$

[Next verbatim source excerpt]

A4. Service Process
 The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$$V_i'(X_i) = p_i + v_i W_i(X_i) \quad (5)$$
 where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of p_i per job, and the expected class- i delay payment of $v_i * W_i(X_i)$.
 The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.
A1. Arrival Processes
 Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and
 The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
 108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
 All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.
 by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded library semantic target

```

type:
{n : ℕ} →
(Fin n → ℝ) →
(i : Fin n) →
AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i → ℝ → ℝ → Option ℝ

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
(z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) (older t : ℝ) =>
AppliedModelingLib.Queueing.nonpreemptivePriorityRecordedCompletionTime
(AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFiniteReplayPostArrivalState meanService i z older t)
(i, 0)
  
```

Exact Lean library declaration

```

/-- The selected customer's completion observation in a fully finite
two-sided replay. -/
noncomputable def stationaryPriorityClassTaggedFiniteReplayPostArrivalResponseTime
{n : ℕ} (meanService : Fin n → ℝ) (i : Fin n)
(z : MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
(older t : ℝ) : Option ℝ :=
nonpreemptivePriorityRecordedCompletionTime
(stationaryPriorityClassTaggedFiniteReplayPostArrivalState meanService i z older t)
(Sigma.mk i 0)
  
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.nonpreemptivePriorityRecordedCompletionTime](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFiniteReplayPostArrivalState](#)

Recorded source-to-library screening Verdict: matches

Reason: This is the selected customer's finite-replay completion observation supporting the Appendix stationary wait bridge.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedPalmMeasure

Source locator: cited publication, lines 227-242

Verbatim paper-source connection

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

AI. Arrival Processes

Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI1 Queue / 873

dependent arrival rate ξ_i . The R arrival processes are stochastically independent.

by the system (per unit of time) to the organization, where we assume for now that the system administra-

[Next verbatim source excerpt]

$L_q^*)$ = The mean number of class- i jobs in the queue in steady state;
 $W_i(\cdot)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i(\cdot)$ = The expected sojourn time of a class- i job in the system in steady state.

KLEINROCK, L. 1967. Optimum Bribing for Queue Position. *Opns. Res.* 15, 304-318.
KLEINROCK, L. 1976. *Queueing System, Vols. I and II.* John Wiley, New York.
KNUDSEN, N. C. 1972. Individual and Social Optimization in a Multi-Server Queue With a General Cost-

Lean-expanded library semantic target

```
type:
{n : ℕ} →
(arrivalRate : Fin n → ℝ) →
(∀ (j : Fin n), 0 < arrivalRate j) →
(i : Fin n) →
MeasureTheory.Measure (AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i)
```

```
value:
fun {n : ℕ} (arrivalRate : Fin n → ℝ) (harrivalRate : ∀ (j : Fin n), 0 < arrivalRate j) (i : Fin n) =>
(AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero
(AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkTaggedArrivalAtZero (harrivalRate i))
(AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkRestLaw arrivalRate harrivalRate i)).Ptag
```

Exact Lean library declaration

```
-- The class-tagged Palm law obtained by inserting one class-`i` arrival at
time zero into the stationary marked-Poisson input. -/
noncomputable def stationaryPriorityClassTaggedPalmMeasure
{n : ℕ} (arrivalRate : Fin n → ℝ)
(harrivalRate : ∀ j, 0 < arrivalRate j) (i : Fin n) :
Measure (MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) :=
(Probability.Palm.targetPassiveTaggedArrivalAtZero
(Probability.Queueing.stationaryPoissonWorkTaggedArrivalAtZero (harrivalRate i))
(multiclassStationaryPoissonWorkRestLaw arrivalRate harrivalRate i)).Ptag
```

Direct retained dependencies

- [AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero](#)
- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero](#)
- [AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkTaggedArrivalAtZero](#)
- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkRestLaw](#)

Recorded source-to-library screening **Verdict:** matches

Reason: This is A1's class-selected arrival-at-zero Palm law; it is routed as a source-model carrier, not as a separate source result.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedResponseTime

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(\cdot)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i(\cdot)$ = The expected sojourn time of a class- i job in the system in steady state.

For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

$$W_i(L1) = W_1 W_{q1} + \sum_{j=1}^i \frac{C_j A_j}{C_i A_i} (A_j - 1) \quad 1131-1140.$$

[Next verbatim source excerpt]

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(\cdot)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay given by $v_i * W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

Al. Arrival Processes
The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.

by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded library semantic target

```
type:
{n : ℕ} →
(Fin n → ℝ) → (i : Fin n) → AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i → ℝ

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n) =>
(a : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) =>
AppliedModelingLib.Probability.stabilizedFiniteReplayResponse
(fun (horizon : ℕ) (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) =>
(AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFiniteReplayPostArrivalResponseTime meanService i z
↑ horizon ↑ horizon).getD
0)
a
```

Exact Lean library declaration

```
/- The selected customer's stationary response is the canonical limsup of
total two-sided finite replays at matching natural past and future horizons.
On the almost-sure coalescence-and-stabilization event below, this is the
literal physical completion epoch. Each finite replay uses zero before the
tag has completed; outside the stabilization event the limsup still provides
a total real-valued version. -/
noncomputable def stationaryPriorityClassTaggedResponseTime
{n : ℕ} (meanService : Fin n → ℝ) (i : Fin n) :
MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i → ℝ :=
Probability.stabilizedFiniteReplayResponse fun horizon z =>
(stationaryPriorityClassTaggedFiniteReplayPostArrivalResponseTime
meanService i z (horizon : ℝ) (horizon : ℝ)).getD 0
```

Direct retained dependencies

- [AppliedModelingLib.Probability.stabilizedFiniteReplayResponse](#)
- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedFiniteReplayPostArrivalResponseTime](#)

Recorded source-to-library screening **Verdict:** matches

Reason: This stabilized finite-replay response observable is infrastructure for the mapped Appendix bridge and is not credited as a standalone source formula.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirement

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process $V_i'(X_i) = \pi_i + v_i W_i$ (5)
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes. where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $v_i * W_i(X)$. The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{Class : Type u_1} →
  (Class → ℝ) →
    (i : Class) →
      ((ℤ → ℝ) × (ℤ → ℝ)) × ({j : Class // j ≠ i} → AppliedModelingLib.Queueing.StationaryPoissonWorkPath) → ℝ

value:
fun {Class : Type u_1} (meanService : Class → ℝ) (i : Class)
  (a : ((ℤ → ℝ) × (ℤ → ℝ)) × ({j : Class // j ≠ i} → AppliedModelingLib.Queueing.StationaryPoissonWorkPath)) =>
    meanService i * AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedRequirement i a
```

Exact Lean library declaration

```
-- The scaled work requirement of a class-`i` customer pinned at time zero
in the concrete multiclass Palm input. -/
def stationaryPriorityClassTaggedWorkRequirement
  (meanService : Class → ℝ) (i : Class) :
    ((ℤ → ℝ) × (ℤ → ℝ)) ×
      ({j : Class // j ≠ i} → StationaryPoissonWorkPath) → ℝ :=
  fun z => meanService i * multiclassStationaryPoissonWorkClassTaggedRequirement i z
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkClassTaggedRequirement](#)

Recorded source-to-library screening Verdict: matches

Reason: It is the selected class's scaled exponential work observable needed by A4's service process.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.stationaryPriorityClassTaggedQueueWait

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(.)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i()$ = The expected sojourn time of a class- i job in the system in steady state.

For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

$$W_i(L1) = W_1 W_{q1} + \sum_{c_j: c_j < c_i} \frac{A_j}{A_i} W_j$$

1131-1140.

[Next verbatim source excerpt]

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay payment of $v_i * W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.

Al. Arrival Processes
Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIMI Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.

by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded library semantic target

```
type:
{n : ℕ} →
(Fin n → ℝ) → (i : Fin n) → AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i → ℝ

value:
fun {n : ℕ} (meanService : Fin n → ℝ) (i : Fin n) :
(a : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i) =>
AppliedModelingLib.Queueing.stationaryPriorityClassTaggedResponseTime meanService i a -
AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirement meanService i a
```

Exact Lean library declaration

```
-- The selected customer's queue-wait observable: its literal response time
minus its own required service. The nonpreemptive trace is the object whose
pathwise workload decomposition will identify this difference with service
completed before the tag starts. -/
noncomputable def stationaryPriorityClassTaggedQueueWait
{n : ℕ} (meanService : Fin n → ℝ) (i : Fin n) :
MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i → ℝ :=
fun z => stationaryPriorityClassTaggedResponseTime meanService i z -
stationaryPriorityClassTaggedWorkRequirement meanService i z
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedResponseTime](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirement](#)

Recorded source-to-library screening Verdict: matches

Reason: Response time minus own service is the source's queue-wait quantity, matching Appendix (A-1) rather than sojourn time.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.stationaryPriorityWorkRequirement

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(\cdot)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay $W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean-expanded library semantic target

```
type:
{Class : Type u_1} →
(Class → ℝ) → Class → (Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) → ℤ → ℝ

value:
fun {Class : Type u_1} (meanService : Class → ℝ) (i : Class)
(a : Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) (a_1 : ℤ) =>
meanService i * AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkRequirement i a a_1
```

Exact Lean library declaration

```
/-- The work requirement of a class-labelled arrival when its class has the
given positive mean service requirement. It scales the concrete unit-rate
exponential mark on the stationary multiclass input carrier. -/
def stationaryPriorityWorkRequirement
(meanService : Class → ℝ) (i : Class) :
(Class → (Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ))) → ℤ → ℝ :=
fun omega n => meanService i *
Probability.PoissonProcess.multiclassStationaryPoissonWorkRequirement i omega n
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkRequirement](#)

Recorded source-to-library screening Verdict: matches

Reason: It supplies A4's class-scaled unit-exponential service-work mark.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

AppliedModelingLib.Queueing.stationaryPriorityWorkRequirement_hasLaw

Source locator: cited publication, lines 264-271

Verbatim paper-source connection

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.

$V_i'(X_i) = \pi_i + v_i W_i$ (5)
where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay given by $v_i * W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

Lean declaration type (metadata)

```
∀ {Class : Type u_1} [inst : Fintype Class] (arrivalRate meanService : Class → ℝ),
  (∀ (i : Class), 0 < arrivalRate i) →
  (∀ (i : Class), 0 < meanService i) →
  ∀ (i : Class) (n : ℤ),
    ProbabilityTheory.HasLaw
      (fun (omega : Class → AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) =>
        AppliedModelingLib.Queueing.stationaryPriorityWorkRequirement meanService i omega n)
      (ProbabilityTheory.expMeasure (meanService i)-1)
      (AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkMeasure arrivalRate)
```

Exact Lean library declaration

```
-- A class-specific scaled work mark has the exponential law whose rate is
-- the reciprocal of its mean service requirement. -/
theorem stationaryPriorityWorkRequirement_hasLaw
  (arrivalRate meanService : Class → ℝ)
  (harrivalRate : ∀ i, 0 < arrivalRate i)
  (hmeanService : ∀ i, 0 < meanService i)
  (i : Class) (n : ℤ) :
  HasLaw (fun omega : Class →
    (Probability.PoissonProcess.GoodSuspensionState × (ℤ → ℝ)) =>
    stationaryPriorityWorkRequirement meanService i omega n)
    (expMeasure (meanService i)-1)
    (Probability.PoissonProcess.multiclassStationaryPoissonWorkMeasure arrivalRate) := by
  have hscale :
    Measure.map (fun x : ℝ => meanService i * x) (expMeasure (1 : ℝ)) =
    expMeasure (meanService i)-1 := by
  calc
    Measure.map (fun x : ℝ => meanService i * x) (expMeasure (1 : ℝ)) =
      Measure.map (fun x : ℝ => x / (meanService i)-1) (expMeasure (1 : ℝ)) := by
        congr 1
        funext x
        field_simp [ne_of_gt (hmeanService i)]
      = expMeasure (meanService i)-1 := by
        exact Probability.map_div_unitExpMeasure_eq_expMeasure
          (inv_pos.mpr (hmeanService i))
  exact HasLaw.comp
    (by fun prop, hscale)
    (Probability.PoissonProcess.multiclassStationaryPoissonWorkRequirement_hasLaw
      arrivalRate harrivalRate i n)
```

Direct retained dependencies

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.multiclassStationaryPoissonWorkMeasure](#)
- [AppliedModelingLib.Queueing.stationaryPriorityWorkRequirement](#)

Recorded source-to-library screening Verdict: matches

Reason: It establishes the A4 exponential work law at reciprocal mean-service rate.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-specific semantic prerequisites

MendelsonWhang1990PriorityPricing.priorityTimeDependentPrice

Paper-local declaration:

MendelsonWhang1990PriorityPricing.priorityTimeDependentPrice

Source locator: cited publication, lines 760-805

Verbatim paper-source connection

and incentive-compatibility in the general setting with heterogeneous service requirements. What price structure is optimal and incentive compatible? The specific PTD pricing scheme we propose is given by $p+(t) = (p+(t), p_2+(t), \dots, p_Z(t))$ with $P_t(t) = A_i t + V_{12} B t^2$ (19) where

$$P_t = \frac{A}{2CR} \sum_{k=1}^R X_k R^2 + \frac{E[p+(TR)]}{R} E[R(R)] - \sum_{k=1}^R S_k - I S_k$$

which completes the proof of optimality.

b. Next we prove incentive-compatibility, that is, class- i users will find it in their best interest to join priority-class i . As in Theorem 2, we prove local optimality and then transitivity.

i. Local optimality: If a class- i job ($i = 1, \dots, R$) joins priority-class i , its expected total cost is $E[p_t((ri))] + v_i * (W_i + c_i)$, where W_i denotes the expected queueing time (excluding service) for priority-class i . On the other hand, if the user chooses priority-class $(i + 1)$, the expected total cost of the job is given by $E[p_l+(Tri)] + v_i * (W_{l+1} + c_i)$. Thus, for all $i = 1, 2, \dots, R - 1$ we have

$$E[p_l+(Tri)] + v_i W_{l+1} - E[p_t+(ri)] + v_i W_i = I$$

with

$$r a, a_i + I A R A R = N_i I 37 + 1 13 i 23 + 1 C I - S$$

$A R = - \frac{a R}{2 S R - 1} * S R$

Lean-expanded paper semantic target

type:
 $\mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R} \rightarrow \mathbb{R}$

value:
fun (linearCoefficient quadraticCoefficient processingTime : $\mathbb{R}$) =>
linearCoefficient * processingTime + 1 / 2 * quadraticCoefficient * processingTime ^ 2

Recorded source-to-declaration screening Verdict: matches

Reason: The expanded Lean value is $A_i t + \text{one-half } B t^2$ squared, exactly Equation (19)'s processing-time-dependent charge for one declared priority class.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient

Paper-local declaration:

MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient

Source locator: cited publication, lines 760-805

Verbatim paper-source connection

and incentive-compatibility in the general setting with heterogeneous service requirements. What price structure is optimal and incentive compatible? The specific PTD pricing scheme we propose is given by $p+(t) = (p+(t), p_2+(t), \dots, p_Z(t))$ with $P_t(t) = A_i t + V_{12} B t^2$ (19)

where

$$B = \frac{E}{k=1} \frac{1}{S_k - I} \cdot k \cdot X_k(20)$$

$A_i 2 + 2 + \dots + 2 = \frac{2(1)}{z-1} \frac{9}{9} k=i+1 \dots k \cdot S_k - 1 S_k$

with $A_i = E L c_i X^*$, $a_i = V_i X A R$, $S_i = R E k=1 \dots R$

So $S = 0$, $S = 1 - S_i$, and where X^* are evaluated at the optimum of Theorem 1, and summations from R to $(R - 1)$ are interpreted as zero (see the Appendix for a summary of the notation). Expression (21) may also be written in the equivalent recursive form

$$A_i = A_i - a_i - 2 + -2 a_i \quad i=2, 3 \dots R$$

with

$$A R = \frac{a R}{S R - 1} \cdot S R$$

$E[pt+(ri)] = E[A_i T_i + 112 B \cdot T_i r] = A_i \cdot c_i + B \cdot c^2$, since $E[r_i] = 2 c_i$. It follows that $p^* = E[pt((ri)]$ for $i=1, 2, \dots, R-1$. Likewise

$$P R = \frac{A}{-2 C R V X k R} \frac{C R + (E \cdot 2 = E[p+(T R)]}{- \cdot R E[R(R)]} \frac{S R - S R k=1 S k - I S k}{S k}$$

which completes the proof of optimality.

b. Next we prove incentive-compatibility, that is, class- i users will find it in their best interest to join priority-class i . As in Theorem 2, we prove local optimality and then transitivity.

i. Local optimality: If a class- i job ($i = 1, \dots, R-1$) joins priority-class i , its expected total cost is $E[pt((ri)] + v_i \cdot (W_i + c_i)$, where W_i denotes the expected queueing time (excluding service) for priority-class i . On the other hand, if the user chooses priority-class $(i + 1)$, the expected total cost of the job is given by $E[p_l+(Tri)] + v_i \cdot (W_{i+1} + c_i)$. Thus, for all $i = 1, 2, \dots, R-1$ we have

$$E L p o(T r i)] + v_i W_i - E E[p+(ri)] + v_i W_{i+1} + I$$

$r a, a_i + I A R$
 $= N i \mid 37 + 1 \mid 3 i 23 + 1 \mid C i - S$
 $_A R$
 $- 1 \mid S [c i v S 3 i + 1 + c i v i + 1 \mid X 1 \mid i - I$

Lean-expanded paper semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:
fun {n : ℕ} (arrivalRate delayCost meanService : Fin n → ℝ) (i : Fin n) =>
MendelsonWhang1990PriorityPricing.priorityLinearNumerator arrivalRate delayCost meanService i /
(AppliedModelingLib.Queueing.finitePriorityStrictSlack meanService arrivalRate i *
AppliedModelingLib.Queueing.finitePriorityInclusiveSlack meanService arrivalRate i ^ 2) +
Σ k : Fin n,
if i < k then
(1 /
(AppliedModelingLib.Queueing.finitePriorityStrictSlack meanService arrivalRate k ^ 2 *
AppliedModelingLib.Queueing.finitePriorityInclusiveSlack meanService arrivalRate k) +
1 /
(AppliedModelingLib.Queueing.finitePriorityStrictSlack meanService arrivalRate k *
AppliedModelingLib.Queueing.finitePriorityInclusiveSlack meanService arrivalRate k ^ 2)) *
MendelsonWhang1990PriorityPricing.priorityLinearNumerator arrivalRate delayCost meanService k
else 0

Direct retained dependencies

- [AppliedModelingLib.Queueing.finitePriorityInclusiveSlack](#)
- [AppliedModelingLib.Queueing.finitePriorityStrictSlack](#)
- [MendelsonWhang1990PriorityPricing.priorityLinearNumerator](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Its strict/inclusive slack terms and lower-priority suffix are Equation (21)'s A_i coefficient.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

--

MendelsonWhang1990PriorityPricing.priorityQueueingTime

Paper-local declaration:

MendelsonWhang1990PriorityPricing.priorityQueueingTime

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(.)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i(.)$ = The expected sojourn time of a class- i job in the system in steady state.
For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)
$$W_i(L1) = W_1 W_{qc1} + \frac{C_i A R}{c_j} (A - c_j) \quad (1)$$

[Next verbatim source excerpt]

KLEINROCK, L. 1976. Queueing System, Vols. I and II. John Wiley, New York.
KNUDSEN, N. C. 1972. Individual and Social Optimization in a Multi-Server Queue With a General Cost-Benefit Structure. *Econometrica* 40, 515-528.
KREPS, D. 1990. A Course in Microeconomic Theory. Princeton University Press, Princeton, N.J.
MARCHAND, M. 1974. Priority Pricing. *Mgmt. Sci.* 20, 1131-1140.

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(.)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.
$$V_i'(X_i) = p_i + v_i W_i(X_i) \quad (5)$$

where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of p_i per job, and the expected class- i delay payment of $v_i * W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.
Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

A1. Arrival Processes
The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIM1 Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.
by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded paper semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (Fin\ n \rightarrow \mathbb{R}) \rightarrow (Fin\ n \rightarrow \mathbb{R}) \rightarrow Fin\ n \rightarrow \mathbb{R}$
value:
 $fun\ \{n : \mathbb{N}\}\ (arrivalRate\ meanService : Fin\ n \rightarrow \mathbb{R})\ (i : Fin\ n) =>$
 $AppliedModelingLib.Queueing.finiteNonpreemptivePriorityQueueWait\ arrivalRate\ meanService\ i$

Direct retained dependencies

- [AppliedModelingLib.Queueing.finiteNonpreemptivePriorityQueueWait](#)

Recorded source-to-declaration screening Verdict: matches

Reason: It names the Appendix (A-1) stationary nonpreemptive-priority M/M/1 queue-wait expression with the source priority ordering.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

MendelsonWhang1990PriorityPricing.homogeneousPriorityQueueingTime

Source locator: cited publication, lines 387-438

Verbatim paper-source connection

Consider a system with R job classes that satisfy and assigning the highest priority (= priority 1) to class 1, the second-highest priority to class 2, and so on, down to class R . In the special case of equal service requirements, $c_i = C_2 = \dots = C_R$, and priorities are determined by the ranking of delay costs. This policy of generality. In this case, jobs are characterized by their delay cost parameters, and the optimal non-preemptive priority scheme orders jobs by decreasing delay costs (i.e., $v_{i1} > v_{i2} > \dots > v_{iR}$), assigning a higher priority to job classes i with a higher delay cost v_i . Under this scheme, the expected number of class- i jobs and the expected waiting time in the system for a class- i job at X are, respectively, given by equation (A- 1) in the Appendix, namely

$$X_i S R$$

the c -A rule. The net value maximization problem

$$N_i - I * S_i S + X_i S \quad S/-i =$$

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

PriorityPricingfortheMIM/1 Queue / 875

and attempt to maximize the objective function of the organization as a whole. Specifically, a user whose job belongs to class i may find it in his self-interest to classify a job into a different priority class, j , either in order

$$W_i(S) = - \sum_{j=1}^R W_j(X) + 1 \quad (9)$$

where $S_0 = 0$, $S_i = E L X_k$, S_i to gain

= 1 priority

- S_i and reduce for the all expected i delay = 1,

2, 3,..., R and W_{iq} denotes cost the when $j < i$, or to reduce the service charge expected waiting timewhen

$j > i$. This would result in a divergence between users' of a class- i job in the queue. Also, using (9)

Lean-expanded paper semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:
 $\text{fun } \{n : \mathbb{N}\} \text{ (arrivalRate : Fin } n \rightarrow \mathbb{R}) \text{ (i : Fin } n) =>$
MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate (fun (x : Fin n) => 1) i

Direct retained dependencies

- MendelsonWhang1990PriorityPricing.priorityQueueingTime

Recorded source-to-declaration screening Verdict: matches
Reason: It is the Appendix (A-1) priority queue-wait formula specialized to homogeneous unit mean service.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

MendelsonWhang1990PriorityPricing.homogeneousPriorityPrice

Paper-local declaration:

MendelsonWhang1990PriorityPricing.homogeneousPriorityPrice

Source locator: cited publication, lines 448-460

Verbatim paper-source connection

Using Theorem 1, the optimal prices are given by

$$R \sum_{k=1}^{\infty} X_k V_k$$

$$+ \sum_{k=1}^{\infty} E_k W_k + \sum_{k=1}^{\infty} X_{k+1} V_{k+1} W_k + \sum_{k=1}^{\infty} O_k$$

In this section, we consider the family of priority-dependent (PD) pricing schemes, where each job is charged only on the basis of the priority class selected.

A PD pricing scheme is specified by a price vector $P = (P_1, P_2, \dots, P_R)$; for a given p , the arrival-rate vector and priority assignment $(X(p), r(p))$ constitute

where we define $X_{A+1} = V_{R+1} = W_R = 0$.
Equation (10) clearly demonstrates the delay cost-

an (symmetric Nash) equilibrium if each class- i user's action maximizes the expected net value, taking all

Lean-expanded paper semantic target

type:

$\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:

$\text{fun } \{n : \mathbb{N}\} (\text{arrivalRate delayCost} : \text{Fin } n \rightarrow \mathbb{R}) (i : \text{Fin } n) =>$

```
  Σ k : Fin n,
    arrivalRate k * delayCost k /
    (AppliedModelingLib.Queueing.finitePriorityInclusiveSlack (fun (x : Fin n) => 1) arrivalRate k *
     AppliedModelingLib.Queueing.finitePriorityStrictSlack (fun (x : Fin n) => 1) arrivalRate k) +
  Σ k : Fin n,
    if i ≤ k then
      (arrivalRate k * delayCost k * MendelsonWhang1990PriorityPricing.homogeneousPriorityQueueingTime arrivalRate k +
       MendelsonWhang1990PriorityPricing.prioritySuccessorOrZero
        (fun (j : Fin n) =>
          arrivalRate j * delayCost j *
          MendelsonWhang1990PriorityPricing.homogeneousPriorityQueueingTime arrivalRate j)
        k) /
      AppliedModelingLib.Queueing.finitePriorityInclusiveSlack (fun (x : Fin n) => 1) arrivalRate k
    else 0
```

Direct retained dependencies

- [AppliedModelingLib.Queueing.finitePriorityInclusiveSlack](#)
- [AppliedModelingLib.Queueing.finitePriorityStrictSlack](#)
- [MendelsonWhang1990PriorityPricing.homogeneousPriorityQueueingTime](#)
- [MendelsonWhang1990PriorityPricing.prioritySuccessorOrZero](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Its expanded two-sum expression implements Equation (10), including the source's zero-padded successor convention.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

MendelsonWhang1990PriorityPricing.homogeneousPrioritySojournTime

Source locator: cited publication, lines 387-438

Verbatim paper-source connection

Consider a system with R job classes that satisfy and assigning the highest priority (= priority 1) to class 1, the second-highest priority to class 2, and so on, down to class R . In the special case of equal service requirements, $c_1 = c_2 = \dots = c_R$, and priorities are determined by the ranking of delay costs. This policy of generality. In this case, jobs are characterized by their delay cost parameters, and the optimal non-preemptive priority scheme orders jobs by decreasing delay costs (i.e., $v_1 > v_2 > \dots > v_R$), assigning a higher priority to job classes i with a higher delay cost v_i . Under this scheme, the expected number of class- i jobs and the expected waiting time in the system for a class- i job at X are, respectively, given by equation (A- 1) in the Appendix, namely

$$N_i = \frac{S_i}{S - X_i} \quad (8)$$

the c - A rule. The net value maximization problem

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

PriorityPricingfortheMIM/1 Queue / 875

and attempt to maximize the objective function of the organization as a whole. Specifically, a user whose job belongs to class i may find it in his self-interest to classify a job into a different priority class, j , either in order

$$W_i(S) = - \sum_{j=1}^R W_j(X) + 1 \quad (9)$$

where $S_0 = 0$, $S_i = \sum_{j=1}^i X_j$, S_i to gain priority

- S_i and reduce for the all expected i delay = 1,

2, 3,..., R and $W_i(q)$ denotes cost the when $j < i$, or to reduce the service charge waiting timewhen $j > i$. This would result in a divergence between users' of a class- i job in the queue. Also, using (9)

Lean-expanded paper semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:
 $\text{fun } \{n : \mathbb{N}\} \text{ (arrivalRate : Fin } n \rightarrow \mathbb{R}) \text{ (i : Fin } n) =>$
MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate (fun (x : Fin n) => 1) i

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.prioritySojournTime](#)

Recorded source-to-declaration screening Verdict: matches
Reason: Equation (9) is the homogeneous queue wait plus the unit service requirement.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost

Paper-local declaration:

MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost

Source locator: cited publication, lines 448-460

Verbatim paper-source connection

Using Theorem 1, the optimal prices are given by

$$R \sum_{k=1}^K X_k V_k$$

$$+ \sum_{k=1}^K \frac{X_{k+1} V_{k+1} W_k q + (0)}{O_k}$$

In this section, we consider the family of priority-dependent (PD) pricing schemes, where each job is charged only on the basis of the priority class selected. A PD pricing scheme is specified by a price vector $P = (P_1, P_2, \dots, P_R)$; for a given p , the arrival-rate vector and priority assignment $(\lambda(p), r(p))$ constitute

where we define $X_{A+1} = V_{R+1} = W_R = 0$.
Equation (10) clearly demonstrates the delay cost-

an (symmetric Nash) equilibrium if each class- i user's action maximizes the expected net value, taking all

Lean-expanded paper semantic target

type:

$\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:

```
fun {n : ℕ} (arrivalRate delayCost : Fin n → ℝ) (i selectedPriority : Fin n) =>
  MendelsonWhang1990PriorityPricing.homogeneousPriorityPrice arrivalRate delayCost selectedPriority +
  delayCost i * MendelsonWhang1990PriorityPricing.homogeneousPrioritySojournTime arrivalRate selectedPriority
```

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.homogeneousPriorityPrice](#)
- [MendelsonWhang1990PriorityPricing.homogeneousPrioritySojournTime](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The Section 2 private priority cost is Equation (10)'s declared-priority price plus true-class delay cost times the declared-priority sojourn time.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

MendelsonWhang1990PriorityPricing.homogeneousPriorityPopulation

Source locator: cited publication, lines 387-438

Verbatim paper-source connection

Consider a system with R job classes that satisfy and assigning the highest priority (= priority 1) to class 1, the second-highest priority to class 2, and so on, down to class R . In the special case of equal service requirements, $c_i = c_2 = \dots = c_R$, and priorities are determined by the ranking of delay costs. This policy of generality. In this case, jobs are characterized by their delay cost parameters, and the optimal non-preemptive priority scheme orders jobs by decreasing delay costs (i.e., $v_1 > v_2 > \dots > v_R$), assigning a higher priority to job classes i with a higher delay cost v_i . Under this scheme, the expected number of class- i jobs and the expected waiting time in the system for a class- i job at X are, respectively, given by equation (A- 1) in the Appendix, namely

$$N_i = \frac{S_i}{S - X_i} \quad (8)$$

the c - A rule. The net value maximization problem

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

PriorityPricingfortheMIM/1 Queue / 875

and attempt to maximize the objective function of the organization as a whole. Specifically, a user whose job belongs to class i may find it in his self-interest to classify a job into a different priority class, j , either in order

$$W_i(S) = - \sum_{j=1}^R W_j(X) + 1 \quad (9)$$

where $S_0 = 0$, $S_i = \text{EL } X_k$, S_i to gain = 1 priority - S_i and reduce for the all expected i delay = 1,

2, 3, ..., R and W_{iq} denotes cost the when $j < i$, or to reduce the service charge expected waiting timewhen $j > i$. This would result in a divergence between users' of a class- i job in the queue. Also, using (9)

Lean-expanded paper semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:
 $\text{fun } \{n : \mathbb{N}\} (\text{arrivalRate} : \text{Fin } n \rightarrow \mathbb{R}) (i : \text{Fin } n) =>$
 $\text{arrivalRate } i * \text{MendelsonWhang1990PriorityPricing.homogeneousPrioritySojournTime arrivalRate } i$

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.homogeneousPrioritySojournTime](#)

Recorded source-to-declaration screening Verdict: matches

Reason: Equation (8) is the homogeneous Little's-law population: class arrival rate multiplied by its sojourn time.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

MendelsonWhang1990PriorityPricing.individualDemandCondition

Paper-local declaration:

MendelsonWhang1990PriorityPricing.individualDemandCondition

Source locator: cited publication, lines 258-268

Verbatim paper-source connection

relationship (1). Let the system administrator set the

A3. Delay Cost Structure

R-dimensional price vector $p = (P_1, P_2, P_3, \dots, P_R)$,

Each class- i job is characterized by a delay cost of v_i where p_i is the price charged to a class- i job. Then, the per unit time. class- i demand relationship is given by

A4. Service Process

$V_i'(X_i) = p_i + v_i W_i$ (5)

The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(-)$ of a class- i job, r_i , follows an exponential distribution with finite cost, given by $v_i * W_i(X)$. where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of p_i per job, and the expected class- i delay

[Next verbatim source excerpt]

A5. Decision Structure

price per class- i job is given by

We assume the individual decision structure: users do not collude, and each maximizes his expected net

R aow.

$P_i = v_j X_i$, for $i = 1, 2, \dots, R$ (6)

benefit.

Lean-expanded paper semantic target

type:

{Class : Type u_1} →

[Fintype Class] →

(Class → ℝ → ℝ) → (Class → ℝ) → (Class → ℝ) → (Class → ℝ) → (Class → (Class → ℝ) → ℝ) → Class → ℝ → Prop

value:

fun {Class : Type u_1} [Fintype Class] (value : Class → ℝ → ℝ) (delayCost flow price : Class → ℝ)

(waitingTime : Class → (Class → ℝ) → ℝ) (i : Class) (marginalValue : ℝ) =>

HasDerivAt (value i) marginalValue (flow i) ∧ marginalValue = price i + delayCost i * waitingTime i flow

Recorded source-to-declaration screening Verdict: matches

Reason: The Lean predicate records both parts of the source condition: the displayed number is the derivative of class i 's value at its flow, and Equation (5) equates it to direct price plus per-time delay cost times expected delay. This is the atomistic A5 decision equation, not a collusive first-order condition.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

MendelsonWhang1990PriorityPricing.netValue

Paper-local declaration:

MendelsonWhang1990PriorityPricing.netValue

Source locator: cited publication, lines 195-200

Verbatim paper-source connection

Two alternative decision structures may be envisioned in this setting. Either users set the aggregate arrival rate collusively, acting in effect as a single decision maker; or users make individual decisions on whether or not to join the system, and the overall arrival rate is determined by aggregating the results of

and the second represents the expected delay cost. p, represented by the demand relationship (1), we turn to the problem of maximizing the expected net value of the system as a whole. The net value of the system at arrival rate X is given by $V(X) - v * X * W(X)$, where the first term represents its expected gross value,

[Next verbatim source excerpt]

The value function of the class- i job stream can be expressed as $V_i(X_i)$, and the total value is given by the sum of $V_i(X_i)$ over all classes, that is, the expected (gross) value function is given by

belongs. Under a given priority scheme, this problem can be formulated as

$$\max_X \sum_{i=1}^R V_i(X_i) - E \sum_{i=1}^R v_i L_i(X_i) \quad (4)$$

$V(X) = E \sum_{i=1}^R V_i(X_i)$ where L_i is the mean number of class- i jobs in the system in steady state, $v_i L_i$ is the expected delay cost

incurred by class- i jobs per unit of time, and $V(2X) = E \sum_{i=1}^R V_i(2X_i)$ is the total gross value per unit of time.

Lean-expanded paper semantic target

```
type:
{Class : Type u_1} → [Fintype Class] → (Class → ℝ → ℝ) → (Class → ℝ) → (Class → (Class → ℝ) → ℝ) → (Class → ℝ) → ℝ

value:
fun {Class : Type u_1} [Fintype Class] (value : Class → ℝ → ℝ) (delayCost : Class → ℝ)
  (waitingTime : Class → (Class → ℝ) → ℝ) (flow : Class → ℝ) =>
  AppliedModelingLib.finiteDelayNetValue value delayCost waitingTime flow
```

Direct retained dependencies

- [AppliedModelingLib.finiteDelayNetValue](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The expanded finite sum is gross value $V_i(\lambda_i)$ less $v_i \lambda_i W_i(\lambda_i)$ for every class. Since $\lambda_i W_i$ is the steady-state class population by Little's law, this is exactly Equation (4)'s total value minus total delay cost.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

MendelsonWhang1990PriorityPricing.priorityCheatingPenalty

Source locator: cited publication, lines 921-937

Verbatim paper-source connection

arrive during the service time of the tagged job totals $v_1 X t^2/2$. The impact of this delayed startup of the class- i busy period propagates to all class- i jobs that arrive by the end of the busy period, and the total externality cost inflicted on class- i jobs is given by

$$= 2 \left(\frac{X i t}{2} + \frac{C i X^2 t}{2} + C^2 X^3 t + \dots \right) v_i X t^2$$

The externality E_k on class $(k \neq i)$ by the tagged job can be computed in a similar way, but each service period must now be extended by a factor of $|k-i|$ because the service process is subject to interruptions

where $l_i'(k) = v_i(W_k - W_i) + c_i(A_k - A_i)$. The term $l_i(k)$ can be regarded as the cheating penalty incurred when a class- i user chooses priority-class k . Theorem 3 demonstrates that $l_i'(k)$, as a discrete function of k , has a unique minimum value of zero at $k = i$. Hence, any class- i user choosing a different priority class will incur a strictly positive penalty. Another desirable property of the cheating penalty $l_i'(k)$, given by the following theorem, is that for all user classes i , the cheating penalty $l_i'(k)$ increases as the selected priority-class, k , gets farther away from the correct user class, i , in either direction. That is, the cheating penalty increases with the degree of the cheating.

Lean-expanded paper semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Fin } n \rightarrow \text{Fin } n \rightarrow \mathbb{R}$

value:
`fun {n : ℕ} (meanService delayCost queueingTime linearCoefficient : Fin n → ℝ) (customer declaredPriority : Fin n) =>
 meanService customer * (linearCoefficient declaredPriority - linearCoefficient customer) +
 delayCost customer * (queueingTime declaredPriority - queueingTime customer)`

Recorded source-to-declaration screening Verdict: matches
Reason: The Lean value is $c_i(A_k - A_i) + v_i(W_k - W_i)$, exactly the source's displayed cheating penalty for a true class i declaring priority k . The common expected quadratic component cancels, leaving the linear-price and queue-wait differences shown in the source.
Reviewer check: ☐ Matches source input
If left unchecked, explain the discrepancy or uncertainty below.
Reviewer annotation

where X^* achieves the overall optimum. and (18)

[Next verbatim source excerpt]

Consider a system with R job classes that satisfy class- i jobs. As before, all aggregate class characteristics assumptions A1-A5 of Section 1, where class- i jobs are common knowledge, whereas individual job characteristics are known only to the user. Thus, both the c_i . We assume that

Lean-expanded paper semantic target

```
type:
{n : ℕ} → (Fin n → ℝ) → (Fin n → ℝ) → (Fin n → ℝ) → (Fin n → ℝ → ℝ) → Fin n → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate delayCost meanService : Fin n → ℝ) (pricing : Fin n → ℝ → ℝ)
  (customer declaredPriority : Fin n) =>
  ∫ (processingTime : ℝ),
    pricing declaredPriority processingTime ∂ProbabilityTheory.expMeasure (meanService customer)-1 +
    delayCost customer *
    (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService declaredPriority +
     meanService customer)
```

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.priorityQueueingTime](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The selected source bundle defines the PTD equilibrium by comparing $E[p_j(T_i)] + v_i W_j^i(X(p), r(p))$ across priority classes j , identifies T_i as class i 's actual processing requirement, calls W the expected total time in system, and states that class- i processing requirements are exponential with mean c_i . The displayed Lean target integrates the declared-priority charge against $\exp\text{Measure}(\text{meanService customer})^{-1}$ and adds $\text{delayCost customer}$ times the declared-priority queueing time plus that customer's mean service time. Its displayed queueing and sojourn prerequisites make the latter the same total-time component, so the formula preserves the source's direct-charge and delay-cost terms on the displayed positive-mean stationary uses.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain

Paper-local declaration:

MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain

Source locator: cited publication, lines 1118-1129

Verbatim paper-source connection

$W_i(\cdot)$ = The expected waiting time of a class- i job in the queue in steady state;
 $W_i(\cdot)$ = The expected sojourn time of a class- i job in the system in steady state.
For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)
$$W_i(L1) = W_1 W_{q1} + \frac{C_i A_i}{c_j} (A_i - c_j) \quad (1)$$

[Next verbatim source excerpt]

KLEINROCK, L. 1976. Queueing System, Vols. I and II. John Wiley, New York.
KNUDSEN, N. C. 1972. Individual and Social Optimization in a Multi-Server Queue With a General Cost-Benefit Structure. *Econometrica* 40, 515-528.
KREPS, D. 1990. A Course in Microeconomic Theory. Princeton University Press, Princeton, N.J.
MARCHAND, M. 1974. Priority Pricing. *Mgmt. Sci.* 20, 1131-1140.

A4. Service Process
The processor is modeled as an M/M/1 queueing system with nonpreemptive priority among job classes. The processing requirement $H_i(\cdot)$ of a class- i job, r_i , follows an exponential distribution with finite mean c_i . Processing requirements are independent across jobs and identically distributed within job classes.
$$V_i'(X_i) = \pi_i + v_i W_i(X_i) \quad (5)$$

where (5) is analogous to (1) with total expected class- i user cost consisting of two components: a direct payment of π_i per job, and the expected class- i delay payment of $v_i * W_i(X_i)$.
The following theorem generalizes the net value maximizing pricing scheme, (3), to the multiclass case.

[Next verbatim source excerpt]

giving rise to $X = A(p)$ or $V'(X) = 4 \cdot (X/A) = p$.
Consider the case where the delay cost v is positive. Here, the cost faced by an individual user consists of two components: a direct payment of p per job, and

A1. Arrival Processes
The arrival process of class- i jobs to the system, which reflects the aggregation of infinitesimal users' job flow, follows a stationary Poisson process with a price-

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]

Priority Pricing for the MIM1 Queue / 873

dependent arrival rate X_i . The R arrival processes are stochastically independent.
by the system (per unit of time) to the organization, where we assume for now that the system administra-

Lean-expanded paper semantic target

type:
 $\{n : \mathbb{N}\} \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow (\text{Fin } n \rightarrow \mathbb{R}) \rightarrow \text{Prop}$
value:
 $\text{fun } \{n : \mathbb{N}\} (\text{meanService } a : \text{Fin } n \rightarrow \mathbb{R}) \Rightarrow$
 $\{ \text{flow} : \text{Fin } n \rightarrow \mathbb{R} \mid$
 $\quad (\forall (j : \text{Fin } n), 0 < \text{meanService } j) \wedge (\forall (j : \text{Fin } n), 0 \leq \text{flow } j) \wedge \sum j : \text{Fin } n, \text{flow } j * \text{meanService } j < 1 \}$
 $\quad a$

Recorded source-to-declaration screening Verdict: matches

Reason: cited publication:1118-1129 gives the stationary nonpreemptive-priority M/M/1 time formula with S_i as cumulative offered work and its slack as $1 - S_i$; the target is precisely the nonnegative-rate, positive-mean, total-load-below-one stationary domain needed for that formula.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

MendelsonWhang1990PriorityPricing.priorityTimeDependentPricingOptimalAndIncentiveCompatibleAt

Source locator: cited publication, lines 326-362

Verbatim paper-source connection

Hence becomes

$$\begin{aligned} & \max_j \sum_{i=1}^R V_i(X_i) - v_i L_i(r, t) \\ & \text{If } X^* \text{ and } r^* \text{ are the optimal arrival-rate vector and scheduling permutation, respectively, then by the optimality of the c-, rule for fixed } X = X^* \text{ we have} \\ & \max\{V_i(X_i) - v_i L_i(X, r)\} \\ & = E \sum_{i=1}^R V_i(X_i) - E \sum_{i=1}^R v_i L_i(2^*, r^*) \end{aligned}$$

1a-

$$\begin{aligned} & \sum_{i=1}^R \lambda_i (2X) a - \sum_{i=1}^R \lambda_i a \\ & = -i a' J/(X). \end{aligned}$$

Note that when $R = 1$, (6) reduces to (3). Also, as in (3), the right-hand side of (6) represents the expected cost inflicted on all other users as a consequence of an infinitesimal unit increase in the job flow of a class- i user. Thus, similar to (3), we may call the right-hand side of (6) the externality cost. The result of Theorem

[Next verbatim source excerpt]

Consider a system with R job classes that satisfy class- i jobs. As before, all aggregate class characteristics are common knowledge, whereas individual job characteristics are known only to the user. Thus, both the c_i . We assume that

[Next verbatim source excerpt]

Under this general framework, an equilibrium can be defined in a similar way to (12) and (13). Let W_j denote the expected total waiting time in the system for a class- i job picking priority-class j . Then, $(X(p), P_2(t))$ be given by (16). If $p(t)$ is optimal and incentive-compatible, optimality implies that for a class- i job, scheme $p(t)$ if for each $i = 1, 2, \dots, R$ we just have $E[p_i(r_i)] = p^*$, where p^* are given by Theorem 1. Also from (16)

$$\begin{aligned} & E[p_r(p)(r_i)] + v_i W_i(p)(2(p), r(p)) \\ & = \min E[p_j(r_i)] + v_i W_j(X(p), r(p)) \end{aligned}$$

and

$$V_i'(X_i(p)) = E[p_r(p)(T_i)] + v_i W_r(p)(X(p), r(p))$$

where T_i is the actual processing requirement of a class- i job.

In order for the pricing schedule $p(t)$ to be incentive-compatible, it should satisfy the following conditions:

1. Incentive-compatibility: For all $i = 1, 2, \dots, R$ and $r_i(p?) = i$ (14)
2. Optimality: The resulting arrival Hence using (17), incentive-compatibility implies the expected net value of the system is, for all $i = 1, 2, \dots, R$

$$X(p_0) = \sum_{i=1}^R \lambda_i X_i^* \quad (15)$$

where X^* achieves the overall optimum. and (18)

[Next verbatim source excerpt]

Consider a system with R job classes that satisfy class- i jobs. As before, all aggregate class characteristics assumptions A1-A5 of Section 1, where class- i jobs are common knowledge, whereas individual job characteristics are known only to the user. Thus, both the ci. We assume that

Lean-expanded paper semantic target

```
type:
{n : ℕ} →
(Fin n → ℝ → ℝ) →
(Fin n → ℝ) →
(Fin n → ℝ) →
(meanService : Fin n → ℝ) →
(Fin n → ℝ) → (∀ (j : Fin n), 0 < meanService j) → (Fin n → ℝ → ℝ) → (Fin n → Fin n) → Prop

value:
fun {n : ℕ} (value : Fin n → ℝ → ℝ) (arrivalRate delayCost meanService marginalValue : Fin n → ℝ)
(hmeanService : ∀ (j : Fin n), 0 < meanService j) (pricing : Fin n → ℝ → ℝ) (assignedPriority : Fin n → Fin n) =>
IsMaxOn
(fun (pair : (Fin n → ℝ) × Equiv.Perm (Fin n)) =>
MendelsonWhang1990PriorityPricing.priorityScheduleNetValue value pair.1 delayCost meanService pair.2)
((MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain meanService).prod Set.univ)
(arrivalRate, Equiv.refl (Fin n)) ∧
(∀ (i : Fin n),
HasDerivAt (value i) (marginalValue i) (arrivalRate i) ∧
marginalValue i =
MendelsonWhang1990PriorityPricing.priorityTimeDependentExpectedCost arrivalRate delayCost meanService
pricing i (assignedPriority i)) ∧
(∀ (i selectedPriority : Fin n),
MendelsonWhang1990PriorityPricing.priorityTimeDependentExpectedCost arrivalRate delayCost meanService pricing
i (assignedPriority i) ≤
MendelsonWhang1990PriorityPricing.priorityTimeDependentExpectedCost arrivalRate delayCost meanService
pricing i selectedPriority) ∧
∀ (i : Fin n), assignedPriority i = i
```

Direct retained dependencies

- [MendelsonWhang1990PriorityPricing.priorityScheduleNetValue](#)
- [MendelsonWhang1990PriorityPricing.priorityTimeDependentExpectedCost](#)
- [MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain](#)

Recorded source-to-declaration screening Verdict: matches

Reason: The selected source bundle states the equilibrium cost-minimization condition over every priority j and the marginal-user equality $V_i'(X_i(p)) = E[p \{r_i(p)\}(T_i)] + v_i W \{r_i(p)\}^i(X(p), r(p))$; it then requires incentive compatibility $r_i(p^0) = i$ and optimality $X(p^0) = X^*$, where X^* achieves the overall optimum. The displayed Lean target jointly maximizes the displayed stationary net-value objective over stable flow vectors and priority permutations at $(\text{arrivalRate}, \text{identity})$, requires the derivative/equal-expected-cost equality for each class, minimizes that expected cost over every selected priority, and requires assignedPriority $i = i$. The displayed stable-flow and priority-sojourn prerequisites restrict this comparison to the source's exponential stationary priority-queue domain, so all source conditions in the supplied bundle are represented without a changed formula, quantifier, or conclusion.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

MendelsonWhang1990PriorityPricing.sourceDelayCostConditions

Paper-local declaration:

MendelsonWhang1990PriorityPricing.sourceDelayCostConditions

Source locator: cited publication, lines 259-262

Verbatim paper-source connection

A3. Delay Cost Structure

Each class- i job is characterized by a delay cost of v_i per unit time. R -dimensional price vector $p = (P_1, P_2, P_3, \dots, P_R)$, where p_i is the price charged to a class- i job. Then, the class- i demand relationship is given by

Lean-expanded paper semantic target

type:
{Class : Type u_1} → [Fintype Class] → (Class → ℝ) → Prop

value:
fun {Class : Type u_1} [Fintype Class] (delayCost : Class → ℝ) => ∀ (i : Class), 0 ≤ delayCost i

Recorded source-to-declaration screening Verdict: matches

Reason: The source calls v_i a delay cost per unit time and explicitly introduces the one-class delay cost as positive. Lean records the corresponding nonnegative cost-domain convention class by class, including the zero-cost boundary; this weak closure does not omit or reverse any source restriction and makes the formal theorem at least as general as the stated cost model.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions

Paper-local declaration:

MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions

Source locator: cited publication, lines 244-257

Verbatim paper-source connection

A2. Value Function

The value function of the class- i job stream can be expressed as $V_i(X_i)$, and the total value is given by the sum of $V_i(X_i)$ over all classes, that is, the expected (gross) value function is given by

belongs. Under a given priority scheme, this problem can be formulated as

$$\max_{X} \sum_{i=1}^R V(X) - \sum_{i=1}^R v_i L_i(X) \quad (4)$$

$V(X) = \sum_{i=1}^R V_i(X_i)$ where L_i is the mean number of class- i jobs in the system in steady state, $v_i L_i$ is the expected delay cost

incurred by class- i jobs per unit of time, and $V(2X) = \sum_{i=1}^R V_i(2X_i)$ is the total gross value per unit of time. Consider the multiple class analog of the demand

where $V_i(X_i)$ are differentiable, nondecreasing and concave functions of their arguments X_i for all i .

Lean-expanded paper semantic target

type:

```
{Class : Type u_1} → [Fintype Class] → (Class → ℝ → ℝ) → Prop
```

value:

```
fun {Class : Type u_1} [Fintype Class] (value : Class → ℝ → ℝ) =>
  (∀ (i : Class), DifferentiableOn ℝ (value i) (Set.Ici 0)) ∧
  (∀ (i : Class), MonotoneOn (value i) (Set.Ici 0)) ∧ ∀ (i : Class), ConcaveOn ℝ (Set.Ici 0) (value i)
```

Recorded source-to-declaration screening Verdict: matches

Reason: The expanded Lean predicate now includes all three properties stated in Assumption A2—differentiability, nondecreasingness, and concavity—for every class value function on the nonnegative flow domain. No A2 property is left only in prose or in a theorem-local premise.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Paper-local declaration:

MendelsonWhang1990PriorityPricing.stationaryClassTaggedExpectedPriorityTimeDependentPrice

Source locator: cited publication, lines 760-805**Verbatim paper-source connection**

and incentive-compatibility in the general setting with heterogeneous service requirements. What price structure is optimal and incentive compatible? The specific PTD pricing scheme we propose is given by $p+(t) = (p+(t), p_2+(t), \dots, p_Z(t))$ with $P_t(t) = A_{it} + V_{12} B_{it}^2$ (19)

where

$$B_{ik} = \frac{E_{ik}}{1 - S_{k-1}} \cdot X_k \quad (20)$$

$A_{ik} = E_{ik} - \frac{2}{z-1} \sum_{k=1}^{z-1} \frac{2}{9} \frac{1}{k} \frac{1}{S_{k-1}} \frac{1}{k} \frac{1}{S_{k-1}}$

with $A_i = E_{i1} - \frac{2}{z-1} \sum_{k=1}^{z-1} \frac{2}{9} \frac{1}{k} \frac{1}{S_{k-1}} \frac{1}{k} \frac{1}{S_{k-1}}$

$S_0 = 0, S_i = 1 - S_i$, and where X_k are evaluated at the optimum of Theorem 1, and summations from R to $(R-1)$ are interpreted as zero (see the Appendix for a summary of the notation). Expression (21) may also be written in the equivalent recursive form

$$A_i = A_{i-1} - a_{i-2} + -2a_{i-1} \quad i=2,3 \dots R$$

with

$$A_R = -\frac{a_R}{SR-1} \cdot SR$$

$E[pt+(ri)] = E[A_i T_i + 112B_i * T_i r] = A_i * c_i + B_i * c_i^2$

since $E[r_i] = 2c_i$. It follows that $p^* = E[pt((ri)]$ for all $i=1, 2, \dots, R-1$.

Likewise

$$PR = -\frac{A}{2CR} \frac{V}{XkR} \frac{CR+(E_{i2} - \frac{2}{z-1} \sum_{k=1}^{z-1} \frac{2}{9} \frac{1}{k} \frac{1}{S_{k-1}} \frac{1}{k} \frac{1}{S_{k-1}})}{R} \frac{E[R(R)]}{SR - \sum_{k=1}^{R-1} S_{k-1} S_k}$$

which completes the proof of optimality.

b. Next we prove incentive-compatibility, that is, class- i users will find it in their best interest to join priority-class i . As in Theorem 2, we prove local optimality and then transitivity.

i. Local optimality: If a class- i job ($i=1$), class- i users will find it in their best interest to join priority-class i . On the other hand, if the user chooses priority-class $(i+1)$, the expected total cost of the job is given by $E[p_l+(Tri)] + v_i * (W_{l+1} + c_i)$. Thus, for all $i=1, 2, \dots, R-1$ we have

$$E[p_l+(Tri)] + v_i W_{l+1} - E[p_l+(ri)] + v_i W_i + l$$
Lean-expanded paper semantic target

```

type:
{n : ℕ} → (arrivalRate : Fin n → ℝ) → (Fin n → ℝ) → (Fin n → ℝ) → (∀ (j : Fin n), 0 < arrivalRate j) → Fin n → Fin n → ℝ

value:
fun {n : ℕ} (arrivalRate delayCost meanService : Fin n → ℝ) (harrivalRate : ∀ (j : Fin n), 0 < arrivalRate j)
(customer declaredPriority : Fin n) =>
f (omega :
((ℤ → ℝ) × (ℤ → ℝ)) × ({ j : Fin n // j ≠ customer } → AppliedModelingLib.Queueing.StationaryPoissonWorkPath)),
MendelsonWhang1990PriorityPricing.heterogeneousPriorityTimeDependentPrice arrivalRate delayCost meanService
declaredPriority
(AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirement meanService customer
omega) ∂ (AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero
(AppliedModelingLib.Probability.Queueing.stationaryWorkTaggedArrivalAtZero (harrivalRate customer))
(AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkRestLaw arrivalRate harrivalRate customer)).Ptag

```

Direct retained dependencies

- [AppliedModelingLib.Probability.Palm.targetPassiveTaggedArrivalAtZero](#)
- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Probability.Queueing.TaggedArrivalAtZero](#)
- [AppliedModelingLib.Probability.Queueing.stationaryPoissonWorkTaggedArrivalAtZero](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.multiclassStationaryPoissonWorkRestLaw](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedWorkRequirement](#)
- [MendelsonWhang1990PriorityPricing.heterogeneousPriorityTimeDependentPrice](#)

Recorded source-to-declaration screening Verdict: matches

Reason: cited publication:760-805 specifies $p_i^+(t)=A_i t+(1/2)B t^2$ and evaluates $E[p_i^+(T_i)]$ for the class-specific exponential requirement; the target is that same PTD charge averaged over the stationary class-tagged service requirement, with the declared priority selecting i .

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Source claims

Review row 1

Source locator: cited publication, lines 272-305

Verbatim source input

Theorem 1. Under the above assumptions, the optimal
A5. Decision Structure

price per class- i job is given by
We assume the individual decision structure: users do
not collude, and each maximizes his expected net

benefit.
$$P_i = \sum_{j=1}^R w_j X_{ij}, \text{ for } i = 1, 2, \dots, R \quad (6)$$

The specific assumptions of exponential service
time distributions and nonpreemptive priority among
job classes will be invoked only from the next section;

where W_j is the expected delay of a class- j job, and
the derivatives $\partial W_j / \partial X_i$ are evaluated at $X = X^*$, the
arrival-rate vector that maximizes (4).

the results derived in this section are quite general,
and apply to any service-time distribution and head-
of-the-line priority discipline. We assume throughout

Proof. The i th first-order condition for maximizing

that the joining and priority decisions are made by
atomistic individual users, each maximizing the
expected net benefit. This individual decision struc-

$$V_i(X_i) = \sum_{j=1}^R w_j X_{ij} - \sum_{j=1}^R w_j W_j(X_j) \quad (7)$$

Furthermore, the class- i demand relationship is again
structure is in sharp contrast to the class (or collusive)
given by (5)

decision structure, where each class is represented by
a single decision maker who makes the arrival-rate
and priority decisions for the class so as to maximize

$$V_i(X_i) = P_i + V_i * w_i$$

Combining the above equations we obtain, at
the expected net value of the class as a whole. Clearly,
pricing decisions for our individual decision structure

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Theorem 1's externality-price identity holds for each selected class whose welfare-maximizing arrival rate is strictly positive, under the paper's stated
 $\hookrightarrow$ value, delay-cost, demand, and differentiability conditions.

Archival source anchor: cited publication, lines 272-305

Expanded PaperInterface specification

```
∀ {Class : Type u_1} [inst : Fintype Class] (value : Class → ℝ → ℝ) (delayCost flow price : Class → ℝ)
(waitingTime : Class → (Class → ℝ) → ℝ) (i : Class) (marginalValue : ℝ) (partialWaiting : Class → ℝ),
MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions value →
MendelsonWhang1990PriorityPricing.sourceDelayCostConditions delayCost →
(∀ (j : Class), 0 ≤ flow j) →
0 < flow i →
IsMaxOn (MendelsonWhang1990PriorityPricing.netValue value delayCost waitingTime)
(Set.Ici fun (x : Class) => 0) flow →
MendelsonWhang1990PriorityPricing.individualDemandCondition value delayCost flow price waitingTime i
marginalValue →
(∀ (j : Class),
HasDerivAt (fun (t : ℝ) => waitingTime j (Function.update flow i t)) (partialWaiting j) (flow i)) →
price i = MendelsonWhang1990PriorityPricing.externalityPrice delayCost flow partialWaiting
```

Retained governing declarations

- [MendelsonWhang1990PriorityPricing.externalityPrice](#)
- [MendelsonWhang1990PriorityPricing.individualDemandCondition](#)
- [MendelsonWhang1990PriorityPricing.netValue](#)
- [MendelsonWhang1990PriorityPricing.paperInterfaceClassicalDecidableEq](#)
- [MendelsonWhang1990PriorityPricing.sourceDelayCostConditions](#)
- [MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions](#)

Lean-elaborated claim roles

```
1. parameter: Type u_1
2. parameter: Fintype Class
3. parameter: Class → ℝ → ℝ
4. parameter: Class → ℝ
5. parameter: Class → ℝ
6. parameter: Class → ℝ
7. parameter: Class → (Class → ℝ) → ℝ
8. parameter: Class
9. parameter: ℝ
10. parameter: Class → ℝ
11. assumption: MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions value
12. assumption: MendelsonWhang1990PriorityPricing.sourceDelayCostConditions delayCost
13. assumption: ∀ (j : Class), 0 ≤ flow j
14. assumption: 0 < flow i
15. assumption: IsMaxOn (MendelsonWhang1990PriorityPricing.netValue value delayCost waitingTime) (Set.Ici fun (x : Class) => 0) flow
16. assumption: MendelsonWhang1990PriorityPricing.individualDemandCondition value delayCost flow price waitingTime i marginalValue
17. assumption: ∀ (j : Class), HasDerivAt (fun (t : ℝ) => waitingTime j (Function.update flow i t)) (partialWaiting j) (flow i)
18. conclusion: price i = MendelsonWhang1990PriorityPricing.externalityPrice delayCost flow partialWaiting
```

Lean proof endpoint (built separately)

MendelsonWhang1990PriorityPricing.theoremOneExternalityPricing

Recorded source-to-Spec screening Verdict: matches_approved_corrected target

Reason: Theorem 1's displayed source statement and proof quantify classes $i, j = 1, \dots, R$ and use the finite sum over j , while the target quantifies an arbitrary finite `Class` through `[Fintype Class]` and concludes the same class- i price as the sum of `delayCost j * flow j * partialWaiting j`. The source proof combines the class- i first-order condition with the individual demand equality; the target makes those two displayed ingredients explicit, and the approved corrected-target record expressly restricts the selected optimal coordinate to $0 < \text{flow } i$, the condition under which that ordinary first-order equality is used. No `DecidableEq Class` premise is needed to represent the source's finite class domain: the displayed local `paperInterfaceClassicalDecidableEq` supplies `Classical.decEq` solely for `Function.update` in the coordinate-derivative expression, so its omission from the public target neither changes the finite-class model domain nor adds or removes a source-side economic assumption.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 2

Source locator: cited publication, lines 1123-1130

Verbatim source input

For the M/M/1 queue with nonpreemptive priority discipline (Heyman and Sobel 1982, p. 435)

KREPS, D. 1990. A Course in Microeconomic Theory. Princeton University Press, Princeton, N.J.

MARCHAND, M. 1974. Priority Pricing. Mgmt. Sci. 20,

$$W_{1CX}^{(L1)} = W_1 W_{qc1} + \frac{C_i A R_{(A-)}}{C_j \cdot (-i-)} \quad 1131-1140.$$

Expanded PaperInterface specification

```
∀ {n : ℕ} (arrivalRate meanService : Fin n → ℝ) (harrivalRate : ∀ (j : Fin n), 0 < arrivalRate j),
(∀ (j : Fin n), 0 < meanService j) →
  Σ j : Fin n, arrivalRate j * meanService j < 1 →
    ∀ (i : Fin n),
      f (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i),
        AppliedModelingLib.Queueing.stationaryPriorityClassTaggedResponseTime meanService i
        z ∂AppliedModelingLib.Queueing.stationaryPriorityClassTaggedPalmMeasure arrivalRate harrivalRate i =
        MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate meanService i ∧
        MendelsonWhang1990PriorityPricing.priorityPopulation arrivalRate meanService i =
        arrivalRate i * MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate meanService i ∧
        MendelsonWhang1990PriorityPricing.priorityPopulation arrivalRate meanService i / arrivalRate i =
        MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate meanService i
```

Retained governing declarations

- [AppliedModelingLib.Probability.PoissonProcess.GoodSuspensionState](#)
- [AppliedModelingLib.Probability.PoissonProcess.instMeasurableSpaceGoodSuspensionState](#)
- [AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample](#)
- [AppliedModelingLib.Queueing.StationaryPoissonWorkPath](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedPalmMeasure](#)
- [AppliedModelingLib.Queueing.stationaryPriorityClassTaggedResponseTime](#)
- [MendelsonWhang1990PriorityPricing.priorityPopulation](#)
- [MendelsonWhang1990PriorityPricing.prioritySojournTime](#)

Lean-elaborated claim roles

1. parameter: ℕ
2. parameter: Fin n → ℝ
3. parameter: Fin n → ℝ
4. assumption: ∀ (j : Fin n), 0 < arrivalRate j
5. assumption: ∀ (j : Fin n), 0 < meanService j
6. assumption: Σ j : Fin n, arrivalRate j * meanService j < 1
7. parameter: Fin n
8. conclusion: f (z : AppliedModelingLib.Queueing.MulticlassStationaryPoissonWorkClassTaggedSample (Fin n) i),
AppliedModelingLib.Queueing.stationaryPriorityClassTaggedResponseTime meanService i
z ∂AppliedModelingLib.Queueing.stationaryPriorityClassTaggedPalmMeasure arrivalRate harrivalRate i =
MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate meanService i ∧
MendelsonWhang1990PriorityPricing.priorityPopulation arrivalRate meanService i =
arrivalRate i * MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate meanService i ∧
MendelsonWhang1990PriorityPricing.priorityPopulation arrivalRate meanService i / arrivalRate i =
MendelsonWhang1990PriorityPricing.prioritySojournTime arrivalRate meanService i

Lean proof endpoint (built separately)

MendelsonWhang1990PriorityPricing.appendixPriorityQueueingTime

Recorded source-to-Spec screening Verdict: matches

Reason: The Appendix A-1 source item is the stationary M/M/1 nonpreemptive-priority mean-time formula. The target instantiates that formula with positive class-specific exponential service means and stable offered load; its population and population-to-rate conjuncts only make the corresponding Little-law notation explicit, with positivity making the division valid.

Reviewer check: ☐ Matches source input

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 3

Source locator: cited publication, lines 550-559

Verbatim source input

Theorem 2. Let p^* be the optimal PD price vector given by (10). Then, p^* is incentive-compatible as well. That is

$$\begin{aligned} p^*l + v_i W, CPk^* + v_i W_k & \quad \text{TCQ} = p^*l + v_i W_i - p, l + v_i W_i - l = TC^* \\ & \quad \text{and} \\ & \quad \text{for any } k \in \{1, 2, \dots, R\} \end{aligned}$$
$$TC' = p^* + v_i W_i < p + l + v_i W_i + l = TQ + l.$$

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Theorem 2's strict priority incentive comparison holds on the homogeneous stable full-positive-flow interior domain, so each class strictly prefers its assigned priority to every distinct priority.

Archival source anchor: cited publication, lines 550-559

Expanded PaperInterface specification

```
∀ {n : ℕ} (arrivalRate delayCost : Fin n → ℝ),
  (∀ (j : Fin n), 0 < arrivalRate j) →
  (∀ (j : Fin n, arrivalRate j < 1) →
    (∀ {i k : Fin n}, i < k → delayCost k < delayCost i) →
      ∀ (i selectedPriority : Fin n),
        selectedPriority ≠ i →
          MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost arrivalRate delayCost i i <
            MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost arrivalRate delayCost i selectedPriority)
```

Retained governing declarations

- [MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost](#)

Lean-elaborated claim roles

1. parameter: $\mathbb{N}$
2. parameter: $\text{Fin } n \rightarrow \mathbb{R}$
3. parameter: $\text{Fin } n \rightarrow \mathbb{R}$
4. assumption: $\forall (j : \text{Fin } n), 0 < \text{arrivalRate } j$
5. assumption: $\sum j : \text{Fin } n, \text{arrivalRate } j < 1$
6. assumption: $\forall \{i k : \text{Fin } n\}, i < k \rightarrow \text{delayCost } k < \text{delayCost } i$
7. parameter: $\text{Fin } n$
8. parameter: $\text{Fin } n$
9. assumption: $\text{selectedPriority} \neq i$
10. conclusion: $\text{MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost arrivalRate delayCost } i i < \text{MendelsonWhang1990PriorityPricing.homogeneousPriorityExpectedCost arrivalRate delayCost } i \text{ selectedPriority}$

Lean proof endpoint (built separately)

MendelsonWhang1990PriorityPricing.theoremTwoPriorityIncentiveCompatibility

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: The source's strict assigned-priority comparison for every distinct priority is the Spec's strict homogeneous expected-cost inequality on the recorded stable full-positive homogeneous domain.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 4

Source locator: cited publication, lines 806-846

Verbatim source input

Theorem 3. The PTD pricing scheme (19) is optimal and incentive-compatible.

$$+ v_i S_i (S_{i+1} - S_{i-1})]$$

AR

Proof. a. We first prove the optimality of (19). It is sufficient to prove that for all $i = 1, 2, \dots$, $E[p_i(r_i)] = p_i^*$, where p_i^* are given by Theorem 1. For all $i = 1, 2, \dots, R-1$, we have from Theorem 1

$$s_1 \sim \dots \sim s_{i+1} + X^* + c_{i+1} + X^* \\ *_{i-1} \quad i+ \quad k_i$$

_AR

$$P' \in \bigcap_{k=1}^{\infty} X_k$$

$$= [332 + 1E(-12 + -2_{ak})] \\ i (i k=i+1 S)k-1Sk Ek \bar{k}-1$$

$$* [i+1(c_i v_{i+1} - CE + l v_1) C_k X_k] j <$$

since by construction $c_i v_{i+1} < c_{i+1} v_i$. Thus

$$E[p_i(r_i)] + v_i W < E[p_{i+1}(r_{i+1})] + v_{i+1} W$$

$$k = l k - 1 i 2 \\ -A1 - . c,$$

i.e., any class- i job will prefer priority-class i over priority-class $(i + 1)$.

$$= A_i * c_i + B * c_i ?$$

This content downloaded from
108.30.55.84 on Mon, 24 Aug 2026 10:55:53 UTC
All use subject to <https://about.jstor.org/terms>

[form-feed in source extraction]
880 / Mendelson and Whang

In a similar way, for $i = 2, 3, \dots, R$

which completes the proof of transitivity for $i < j < k$. Transitivity for $i > j > k$ follows in a similar way.

$$\{E[p_i(r_i)] + v_j W - \{E[p_i(r_i)] + v_i W\} j$$

Incentive-compatibility follows by repeated application of the transitivity principle, showing that $TC' < TC'$ for all $i = 1, 2, \dots, R$, $j \in \{1, 2, 5, \dots, R\} - \{i\}$.

$$= a_i - a + l \mid [AR hAR]$$

[Next verbatim source excerpt]

Under this general framework, an equilibrium can be defined in a similar way to (12) and (13). Let W_{ij} denote the expected total waiting time in the system for a class- i job picking priority-class j . Then, $(X(p), f(p))$ is an equilibrium with respect to the PTD pricing scheme $p(t)$ if for each $i = 1, 2, \dots, R$

geneous service requirements. The answer is negative, as the following example demonstrates. Consider the case of two job classes ($R = 2$) and let $p(t) = (p_1(t), p_2(t))$ be given by (16). If $p(t)$ is optimal and incentive-compatible, optimality implies that for a class- i job, we just have $E[p_i(r_i)] = p_i^*$, where p_i^* are given by Theorem 1. Also from (16)

$$E[p_r(p)(r_i)] + v_i W_{ri}(p)(2(p), r(p))$$

$$= \min \{E[p_j(r_i)] + v_i W_{ij}(X(p), r(p))\}$$

$$E[p_2(T)] = -E[p_1(T)] = -P \\ a \quad a$$

and

$$\text{and} \quad (17)$$

$$V_i'(X_i(p)) = E[p_j(p)(T_i)] + v_i W_{ri}(p)(X(p), r(p))$$

$$E[p_1(T_2)] = a * E[p_2(T_2)] = a * 2 \\ \text{where } a = K_1/K_2.$$

where T_i is the actual processing requirement of a class- i job.

In order for the pricing schedule $p(t)$ to be incentive-compatible, it should satisfy

A PTD pricing scheme $p_0(t)$ will be called optimal and incentive-compatible if it satisfies the following conditions:

$$1. \text{ Incentive-compatibility: For all } i = 1, 2, \dots, R \\ r_i(p?) = i \quad (14)$$

The resulting arrival Hence using (17), incentive-compatibility implies the net value of the system

2. Optimality: the expected is, for all $i = 1, 2, \dots, R$

$$X(p_0) = X^* \quad (15)$$

$$p_i + V * 1 S - p_i^* + V * W \\ a$$

where X^* achieves the overall optimum.

$$\text{and} \quad (18)$$

[Next verbatim source excerpt]

Hence

becomes

$$r \quad R \quad R$$

$$\max_j \{V_i(X_i) - v_i L_i(r)\}.$$

$$P \sim E$$

$$d \text{ of } 3ax,$$

$$+ v_i * [W_i(X^*) + x^* W_i(X^*)] - v_i W_i(X^*)$$

$$R \quad a \sim \sim i \quad x$$

$$R \quad R \quad R$$

$$- \sum_{j=1}^J X_j^* - W(X^*)$$

$$\max \{V_i(X_i) - v_i L_i(X, r)\}$$

$$\text{since for } i \text{ d}$$

$$R \quad R$$

$$= E V_i(X_i) - E v_i L_i(2^*, r^*)$$

$$i=1 \quad i=1$$

$$1a-$$

$$Ax. Li(2X) a - xa$$

$$axi \quad iW xi$$

$$= -i a' / (X).$$

$$R \quad R$$

$$X V_i(X^*) - E v_i L_i(X, r_0)$$
 Note that when $R = 1$, (6) reduces to (3). Also, as
 in (3), the right-hand side of (6) represents the expected

$$R \quad R$$

$$< \max V_i(x_i) - v_i L_i(X, r_0)\}$$
 cost inflicted on all other users as a consequence of
 an infinitesimal unit increase in the job flow of a class-
 i user. Thus, similar to (3), we may call the right-hand
 side of (6) the externality cost. The result of Theorem
 which demonstrates that the same scheduling rate is
 also optimal in the current setting.

[Next verbatim source excerpt]

Consider a system with R job classes that satisfy
 class- i jobs. As before, all aggregate class characteristics
 assumptions A1-A5 of Section 1, where class- i jobs
 are common knowledge, whereas individual job char-
 acteristics are known only to the user. Thus, both the
 ci. We assume that

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Theorem 3's priority-and-time-dependent price implements the welfare-maximizing arrival vector and gives every class a strict assigned-priority preference $\hookrightarrow$ on the heterogeneous stationary full-positive-flow interior domain.

Archival source anchor: cited publication, lines 806-846

Expanded PaperInterface specification

```

∀ {n : ℕ} (value : Fin n → ℝ → ℝ) (arrivalRate delayCost meanService marginalValue : Fin n → ℝ),
MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions value →
MendelsonWhang1990PriorityPricing.sourceDelayCostConditions delayCost →
∀ (harrivalRate : ∀ (j : Fin n), 0 < arrivalRate j) (hmeanService : ∀ (j : Fin n), 0 < meanService j),
  Σ j : Fin n, arrivalRate j * meanService j < 1 →
  (∀ {i k : Fin n}, i < k → meanService i * delayCost k < meanService k * delayCost i) →
  (∀ (i : Fin n), HasDerivAt (value i) (marginalValue i) (arrivalRate i)) →
  IsMaxOn
    (fun (flow : Fin n → ℝ) =>
      MendelsonWhang1990PriorityPricing.netValue value delayCost
        (fun (k : Fin n) (rates : Fin n → ℝ) =>
          MendelsonWhang1990PriorityPricing.prioritySojournTime rates meanService k)
        flow)
    (MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain meanService) arrivalRate →
MendelsonWhang1990PriorityPricing.priorityTimeDependentPricingOptimalAndIncentiveCompatibleAt value
  arrivalRate delayCost meanService marginalValue hmeanService
  (MendelsonWhang1990PriorityPricing.heterogeneousPriorityTimeDependentPrice arrivalRate delayCost
    meanService)
  id Λ
  (∀ (i : Fin n),
    MendelsonWhang1990PriorityPricing.stationaryClassTaggedExpectedPriorityTimeDependentPrice
      arrivalRate delayCost meanService harrivalRate i i =
      MendelsonWhang1990PriorityPricing.externalityPrice delayCost arrivalRate
        (MendelsonWhang1990PriorityPricing.prioritySojournDerivative arrivalRate meanService i)) Λ
  ∀ (i selectedPriority : Fin n),
    selectedPriority ≠ i →
    0 <
      MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
        (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
        (MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost
          meanService)
      i selectedPriority
  
```

Retained governing declarations

- [MendelsonWhang1990PriorityPricing.externalityPrice](#)
- [MendelsonWhang1990PriorityPricing.heterogeneousPriorityTimeDependentPrice](#)

- [MendelsonWhang1990PriorityPricing.netValue](#)
- [MendelsonWhang1990PriorityPricing.priorityCheatingPenalty](#)
- [MendelsonWhang1990PriorityPricing.priorityQueueingTime](#)
- [MendelsonWhang1990PriorityPricing.prioritySojournDerivative](#)
- [MendelsonWhang1990PriorityPricing.prioritySojournTime](#)
- [MendelsonWhang1990PriorityPricing.priorityTimeDependentPricingOptimalAndIncentiveCompatibleAt](#)
- [MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient](#)
- [MendelsonWhang1990PriorityPricing.sourceDelayCostConditions](#)
- [MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions](#)
- [MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain](#)
- [MendelsonWhang1990PriorityPricing.stationaryClassTaggedExpectedPriorityTimeDependentPrice](#)

Lean-elaborated claim roles

```

1. parameter:  $\mathbb{N}$ 
2. parameter:  $\text{Fin } n \rightarrow \mathbb{R} \rightarrow \mathbb{R}$ 
3. parameter:  $\text{Fin } n \rightarrow \mathbb{R}$ 
4. parameter:  $\text{Fin } n \rightarrow \mathbb{R}$ 
5. parameter:  $\text{Fin } n \rightarrow \mathbb{R}$ 
6. parameter:  $\text{Fin } n \rightarrow \mathbb{R}$ 
7. assumption: MendelsonWhang1990PriorityPricing.sourceValueFunctionConditions value
8. assumption: MendelsonWhang1990PriorityPricing.sourceDelayCostConditions delayCost
9. assumption:  $\forall (j : \text{Fin } n), 0 < \text{arrivalRate } j$ 
10. assumption:  $\forall (j : \text{Fin } n), 0 < \text{meanService } j$ 
11. assumption:  $\sum j : \text{Fin } n, \text{arrivalRate } j * \text{meanService } j < 1$ 
12. assumption:  $\forall \{i k : \text{Fin } n\}, i < k \rightarrow \text{meanService } i * \text{delayCost } k < \text{meanService } k * \text{delayCost } i$ 
13. assumption:  $\forall (i : \text{Fin } n), \text{HasDerivAt } (\text{value } i) (\text{marginalValue } i) (\text{arrivalRate } i)$ 
14. assumption: IsMaxOn
    (fun (flow :  $\text{Fin } n \rightarrow \mathbb{R}$ ) =>
      MendelsonWhang1990PriorityPricing.netValue value delayCost
      (fun (k :  $\text{Fin } n$ ) (rates :  $\text{Fin } n \rightarrow \mathbb{R}$ ) => MendelsonWhang1990PriorityPricing.prioritySojournTime rates meanService k)
      flow)
    (MendelsonWhang1990PriorityPricing.stablePriorityFlowDomain meanService) arrivalRate
15. conclusion: MendelsonWhang1990PriorityPricing.priorityTimeDependentPricingOptimalAndIncentiveCompatibleAt value arrivalRate
    delayCost meanService marginalValue hmeanService
    (MendelsonWhang1990PriorityPricing.heterogeneousPriorityTimeDependentPrice arrivalRate delayCost meanService) id  $\wedge$ 
    ( $\forall (i : \text{Fin } n),$ 
      MendelsonWhang1990PriorityPricing.stationaryClassTaggedExpectedPriorityTimeDependentPrice arrivalRate delayCost
      meanService harrivalRate i i =
      MendelsonWhang1990PriorityPricing.externalityPrice delayCost arrivalRate
      (MendelsonWhang1990PriorityPricing.prioritySojournDerivative arrivalRate meanService i))  $\wedge$ 
    ( $\forall (i \text{ selectedPriority} : \text{Fin } n),$ 
      selectedPriority  $\neq i \rightarrow$ 
      0 <
      MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
      (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
      (MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost meanService) i
      selectedPriority

```

Lean proof endpoint (built separately)

MendelsonWhang1990PriorityPricing.theoremThreePriorityTimeDependentPricing

Recorded source-to-Spec screening Verdict: matches_approved_corrected_target

Reason: Theorem 3 states that the Equation (19) PTD schedule is optimal and incentive-compatible, and its supplied proof excerpt identifies each truthful expected charge with the Theorem 1 externality price and derives strict preference over every distinct priority. The target expresses those conclusions through the source's equilibrium conditions and the recorded stable full-positive interior scope, without attributing strict zero-flow comparisons to the archival statement.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation

Review row 5

Source locator: cited publication, lines 940-985

Verbatim source input

Theorem 4. For all user classes $i = 1, 2, \dots, R$, job is, therefore, given by

$$\begin{aligned} R & \quad \text{the cheating penalty } \Pi'(k) \text{ satisfies } 0 = H'(i) < \\ E & \quad \text{Ek} = E - V_k \quad \text{Hl} \cdot (i + 1) < H \cdot 1 \cdot (i + 2) < \dots < H' \cdot (R) \text{ and } H' \cdot (l) > \\ & \quad \text{Xkl(Sk-l)} \\ & \quad \text{Hli(2) > 3k)} \\ & \quad \dots > H'(i) = 0. \text{ Equivalently, } \Pi'(k) \text{ is a} \end{aligned}$$

$k = l \quad k = -2$

unimodal function of k .

which is the quadratic term of (19). The linear term of the proposed price schedule can be interpreted in a similar way as the externality cost inflicted on other jobs arriving before the tagged job starts its service: the first term of $A_i t$, $\text{ait}/(S_i - 1)$ is the externality cost inflicted by the tagged job on the jobs of its own classes (i.e., class- i), while

Proof. Suppose that $j > i$. Then, by the definition of $\Pi'(k)$, we have

$$\begin{aligned} & \text{IH} \cdot (j + 1) - \text{Hli}(j) \\ & = v_1(j + 1) - W_j - c_1(A_j - A_j + 1) \quad (24) \end{aligned}$$

But we know from Theorem 3 that

$$\begin{aligned} k = i + l \quad k = l \quad k = S_k - l \\ \text{Hli}(j + 1) = v_j(j + 1) - W_j - c_j(A_j - A_j + 1) > 0 \end{aligned}$$

represents the externality cost inflicted on the lower class jobs. To further decompose the second term, the first part

or

R

$$\begin{aligned} & t, E \quad a_k \\ & k = i + l \quad S_k - l \quad S_k \end{aligned}$$

$$WUq > -W, q > -(A - A_j + 1). \quad (25)$$

It follows from equations (24) and (25) that

is incurred by the lower class jobs which have arrived before the tagged job arrives, and the second part

$R - 1$

$$\text{Hli}(j + 1) - \text{Hli}(j) > c_j(A_j - A_j + 1) > 0$$

$$\begin{aligned} t - E \quad 3 \quad 2 \\ k = i + l \quad A_k - l_k \end{aligned}$$

where the last inequality follows from $v_i/c_i > v_j/c_j$ and

is incurred by the lower class jobs coming after the $A_j > A_j + 1$. This shows that $0 = H'(i) < \text{Hli}(i + 1) <$

... $< \text{Hli}(R)$.
tagged job, but before the tagged job starts its service.

A similar argument for the case $j < i$ implies

Altogether, the price represents the conditional expectation of the tagged job's externality on the system.

$$\text{fli}(l) > Wl(2) > \dots > \text{iii}(i) = 0.$$

From a class- i user's viewpoint, the total expected

Formalized review target The archival source above is not asserted equivalent to this formalized target.

Theorem 4's cheating penalty is zero at the assigned priority and strictly increases away from it on the stationary full-positive-flow interior domain.

Archival source anchor: cited publication, lines 940-985

Expanded PaperInterface specification

```
 $\forall \{n : \mathbb{N}\} \text{ (arrivalRate delayCost meanService : Fin } n \rightarrow \mathbb{R}),$ 
 $(\forall (j : \text{Fin } n), 0 < \text{arrivalRate } j) \rightarrow$ 
 $(\forall (j : \text{Fin } n), 0 < \text{meanService } j) \rightarrow$ 
 $\sum j : \text{Fin } n, \text{arrivalRate } j * \text{meanService } j < 1 \rightarrow$ 
 $(\forall \{i k : \text{Fin } n\}, i < k \rightarrow \text{meanService } i * \text{delayCost } k < \text{meanService } k * \text{delayCost } i) \rightarrow$ 
 $(\forall (i : \text{Fin } n),$ 
 $\text{MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost}$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService})$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost meanService}) i$ 
 $=$ 
 $0) \wedge$ 
 $(\forall \{i j k : \text{Fin } n\},$ 
 $i < j \rightarrow$ 
 $j < k \rightarrow$ 
 $\text{MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost}$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService})$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost}$ 
 $\text{meanService})$ 
 $i j <$ 
 $\text{MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost}$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService})$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost}$ 
 $\text{meanService})$ 
 $i k) \wedge$ 
 $\forall \{i j k : \text{Fin } n\},$ 
 $k < i \rightarrow$ 
 $j < k \rightarrow$ 
 $\text{MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost}$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService})$ 
 $(\text{MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost}$ 
 $\text{meanService})$ 
```

```

i k <
MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
(MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
(MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost
meanService)
i j

```

Retained governing declarations

- [MendelsonWhang1990PriorityPricing.priorityCheatingPenalty](#)
- [MendelsonWhang1990PriorityPricing.priorityQueueingTime](#)
- [MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient](#)

Lean-elaborated claim roles

```

1. parameter: ℕ
2. parameter: Fin n → ℝ
3. parameter: Fin n → ℝ
4. parameter: Fin n → ℝ
5. assumption: ∀ (j : Fin n), 0 < arrivalRate j
6. assumption: ∀ (j : Fin n), 0 < meanService j
7. assumption: ∑ j : Fin n, arrivalRate j * meanService j < 1
8. assumption: ∀ {i k : Fin n}, i < k → meanService i * delayCost k < meanService k * delayCost i
9. conclusion: (∀ (i : Fin n),
  MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
  (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
  (MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost meanService) i i =
  0) ∧
  (∀ {i j k : Fin n},
    i < j →
      j < k →
        MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
        (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
        (MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost meanService) i j <
        MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
        (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
        (MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost meanService) i k) ∧
  ∀ {i j k : Fin n},
    k < i →
      j < k →
        MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
        (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
        (MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost meanService) i k <
        MendelsonWhang1990PriorityPricing.priorityCheatingPenalty meanService delayCost
        (MendelsonWhang1990PriorityPricing.priorityQueueingTime arrivalRate meanService)
        (MendelsonWhang1990PriorityPricing.priorityTimeLinearCoefficient arrivalRate delayCost meanService) i j

```

Lean proof endpoint (built separately)

MendelsonWhang1990PriorityPricing.theoremFourCheatingPenaltyMonotonicity

Recorded source-to-Spec screening Verdict: matches_approved corrected_target

Reason: The source's zero truthful penalty and two strict monotonicity chains are preserved by the Spec. Its full-positive stationary interior domain is the map-pinned correction, with no archival-equivalence claim.

Reviewer check: ☐ Matches formalized target

If left unchecked, explain the discrepancy or uncertainty below.

Reviewer annotation